TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN
----------o0o----------

Bài tập lớn môn học

CÔNG NGHỆ JAVA

Đề tài: Lập Trình Game WHISPER OF THE EMERALD FOREST

Sinh viên thực hiện: Nguyễn Đức Trường_241230872
Nguyễn Khắc Minh_241224482
Trần Hữu Long _241230780
Lớp : CNTT_K65

Hà Nội, tháng 4 năm 2026

Mục Lục

LỜI CẢM ƠN 2
CHƯƠNG 1 TỔNG QUAN VỀ GAME VÀ GIAO DIỆN NGƯỜI 3
1.1 Tổng quan về thế giới trò chơi 3
1.1.1 Lý do chọn đề tài. 3
1.1.2 Giá trị cốt lõi và Mục tiêu hướng đến. 3
1.1.3 Quá Trình Phát Triển Dự Án Game. 4
1.2 Phân tích Chức năng và Yêu cầu Kỹ thuật 5
1.2.1. Phân tích Chức năng Nhân vật. 5
1.2.2 Chức năng của orc. 5
1.2.3 Thách thức Kỹ thuật và Giải pháp Công nghệ. 6
1.2.4 Yêu cầu về dữ liệu và Tài nguyên (Data & Assets Requirements). 6
1.2.5 Yêu cầu về giao diện người dùng (UI/UX Requirements). 7
1.2.6 Quản lý Độ phân giải và Tỷ lệ hiển thị (Screen Scaling & Aspect Ratio). .. 8
1.2.7 Hệ thống Menu và Luồng chuyển cảnh (Navigation & Transition). 9
1.3 Giao diện và quy tắc trò chơi 10
1.3.1 Thiết kế Giao diện người dùng (User Interface Design). 10
1.3.2 Quy tắc trò chơi cốt lõi (Core Game Rules). 11
1.3.3 Quy tắc quản lý Tài nguyên và Vật phẩm. 14
CHƯƠNG 2 KIẾN TRÚC HỆ THỐNG GAME 16
2.1 Sơ đồ tổ chức thư mục (Project Directory Structure) 16
2.2 Mối liên kết giữa các thành phần (System Interconnections) 21
2.2.1. Liên kết giữa Tiled, Assets và Screens. 21
2.2.2. Liên kết giữa Entities và Tiled (Hệ thống va chạm). 21
2.2.3. Liên kết giữa Services và Screens (Tính năng trực tuyến). 21
2.2.4. Liên kết giữa Util và Hệ thống điều khiển. 22
2.3 Đánh giá tính hiệu quả của cấu trúc 22
2.4 Kết nối cơ sở dữ liệu 22
CHƯƠNG 3. CHI TIẾT CÀI ĐẶT VÀ CẤU TRÚC HỆ THỐNG 24
3.1 Phần code nhân vật 24
3.1.1. Thực thể Người chơi (Player) 24
3.1.2. Phần code OrcEnemy. 32
3.1.3 Thực thể Vật phẩm (Health Potion). 38
3.2 Xây dựng Hệ thống Màn hình và Giao diện người dùng 39
3.2.1 Quản lý Màn hình Chính (Start Screen & Game Screen). 40
3.2.2. Hệ thống Lớp phủ Giao diện (UI Overlays). 45
3.2.3. Xử lý Trạng thái Kết thúc (Death Screen & The End Screen). 47
3.3 Hệ thống Quản lý Bản đồ và Va chạm môi trường 50
3.3.1 Cấu trúc dữ liệu TiledMap và Tileset. 50
3.3.2 Kỹ thuật Render tối ưu và Xử lý khung nhìn (Culling). 51
3.4. Hệ thống Tiện ích và Quản lý Tài nguyên (Utilities & Asset Management) 53
3.4.1 Quản lý Tài nguyên và Bộ nhớ (Assets System). 53
3.4.2. Hệ thống Hoạt ảnh và Đồ họa (Animation & GraphicsUtil). 54
3.4.3. Tiện ích Văn bản và Camera (TextUtil & Camera2D). 56
3.5. Hệ thống Quản lý và Xử lý Âm thanh (Audio Engine) 57
3.5.1. Kiến trúc quản lý âm thanh đa tầng. 57
3.5.2. Hệ thống phản hồi thính giác theo trạng thái (Audio-Visual Sync). 60
LỜI KẾT 62
TÀI LIỆU THAM KHẢO 63

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành nhất đến cô Đào Thị Lệ Thủy – giảng viên môn Lập trình Java đã tận tình hướng dẫn và cung cấp nguồn tri thức và tài liệu hữu ích giúp chúng em trong suốt quá trình làm và hoàn thành báo cáo này.

Trong suốt thời gian học môn, chúng em đã được cô truyền đạt cho những kiến thức lý luận quý báu. Thông qua bài tập lớn kết thúc học phần này, chúng em đã có cơ hội tìm hiểu sâu hơn về tư duy lập trình java và cách áp dụng vào thực tế. Đây là những kiến thức vô cùng bổ ích, tạo nên nền tảng vững chắc cho chúng em khi ra trường và làm việc sau này.

Chúng em xin được gửi lời cảm ơn chân thành và sâu sắc nhất đến cô đã chỉ bảo tận tình, tạo điều kiện thuận lợi để chúng em hoàn thành tốt bài tập lớn lần này.

Tuy nhiên, do kiến thức thực tế còn hạn hẹp, thời gian làm bài cũng quá gấp, nên bài báo cáo chắc chắn không tránh khỏi những thiếu sót trong quá trình tìm hiểu, nghiên cứu và trình bày. Chúng em rất mong sẽ nhận được những ý kiến đóng góp tuyệt vời của cô để bài làm được hẳn hoi, hoàn chỉnh hơn!

Chúng em xin chân thành cảm ơn!

CHƯƠNG 1 TỔNG QUAN VỀ GAME VA GIAO DIỆN NGƯỜI DÙNG

1.1 Tổng quan về thế giới trò chơi

1.1.1 Lý do chọn đề tài.

Niềm đam mê lớn nhất dẫn dắt quá trình thực hiện dự án này chính là mong muốn tái hiện lại những cảm giác nguyên bản của dòng game nhập vai đi ngang (Side-scrolling RPG) cổ điển như Contra, Mario… . Đối với những người thuộc thế hệ trước, những trò chơi trên nền tảng 8-bit hay 16-bit không chỉ là giải trí, mà còn là biểu tượng của một kỷ nguyên sáng tạo đầy mê hoặc, nơi mà sự hạn chế về công nghệ được bù đắp bằng lối chơi lôi cuốn và tính thẩm mỹ Pixel Art đặc trưng.

Việc lựa chọn phát triển "The Whisper of the Emerald Forest" không nằm ngoài mục đích khơi lại sự đơn giản nhưng đầy thử thách đó. Đam mê này không chỉ dừng lại ở mức độ hưởng thụ, mà đã chuyển hóa thành động lực nghiên cứu sâu về kỹ thuật để mô phỏng lại các cơ chế vận hành truyền thống. Đó là cảm giác điều khiển được nhân vật với độ phản hồi chính xác, nhịp độ di chuyển dồn dập và những pha xử lý tình huống đòi hỏi sự tập trung cao độ. Để đạt được điều này, chúng em đã tập trung tối ưu hóa hệ thống xử lý va chạm và các thuật toán logic để tạo ra sự mượt mà tương tự như những cỗ máy Arcade huyền thoại ngày trước.

Bên cạnh đó, dự án còn là sự kết hợp giữa kỹ thuật hiện đại và phong cách đồ họa cổ điển. Việc tự xây dựng hệ thống hoạt ảnh (Animation) cũng như tự dựng mô hình map (TileMap) theo từng khung hình thủ công là cách để tôn vinh giá trị của dòng game truyền thống. Niềm yêu thích dành cho sự hoài cổ đã thúc đẩy việc tìm tòi, áp dụng các nguyên lý lập trình thuần túy nhằm tạo ra một thế giới ảo có chiều sâu, nơi em và các thành viên cố gắng tái dựng lại những ký ức tuổi thơ rực rỡ của người chơi bằng cách gợi nhớ những tựa game đầy hoài niệm. Đây chính là kim chỉ nam xuyên suốt, giúp dự án giữ được linh hồn của một trò chơi cổ điển trong một môi trường mặt bằng game tập trung quá nhiều vào đồ họa.

1.1.2 Giá trị cốt lõi và Mục tiêu hướng đến.

Về sản phẩm: Mục tiêu cốt lõi của dự án là xây dựng hoàn chỉnh một trò chơi nhập vai hành động (ARPG) 2D dạng Slide - Scroller chạy trên nền tảng Java Desktop. Sản phẩm tập trung vào việc tối ưu hóa hiệu suất thông qua việc tự thiết lập Game Engine bằng thư viện Java Swing/AWT/Util mà không sử dụng các framework hỗ trợ bên ngoài. Dự án hướng tới việc quản lý chặt chẽ các thành phần kỹ thuật như: hệ

thống Render đa lớp (Background, Entities, UI), cơ chế xử lý va chạm chính xác giữa các thực thể, và tích hợp bộ dữ liệu tài nguyên (Assets) từ hình ảnh nhân vật đến âm thanh môi trường. Kết quả cuối cùng là một phần mềm có cấu trúc mã nguồn sạch, khả năng đóng gói độc lập và vận hành ổn định trên các cấu hình máy tính khác nhau.

Về trải nghiệm người dùng: Dự án đặt trọng tâm vào việc tái hiện cảm giác hoài cổ của các dòng game Arcade kinh điển, nơi sự tương tác trực tiếp và phản hồi tức thì là yếu tố quyết định. Mục tiêu trải nghiệm hướng đến sự mượt mà trong từng thao tác điều khiển nhân vật như di chuyển, nhảy và tung các chuỗi tấn công (Combo). Hệ thống UI/UX được thiết kế tối giản nhưng hiệu quả, giúp người chơi dễ dàng nắm bắt trạng thái nhân vật (HP) và tương tác với các màn chơi. Thông qua việc tìm tòi, khám phá, chiến đấu với các quái vật (Orc), dự án mong muốn mang lại cảm giác chinh phục thử thách, khơi gợi niềm đam mê khám phá thế giới ảo trong một không gian đồ họa Pixel Art đầy tính nghệ thuật.

1.1.3 Quá Trình Phát Triển Dự Án Game.

Trong quá trình phát triển trò chơi, nhóm đã đối mặt với nhiều thách thức đan xen giữa khâu thiết kế sáng tạo và hiện thực hóa kỹ thuật. Trước hết, để xây dựng một nền tảng vững chắc, nhóm đã tập trung vào giai đoạn Concept & Game Design nhằm phân tích ý tưởng, xác định rõ phong cách hình ảnh và bối cảnh. Từ đó, việc phác thảo World Layout (bản đồ tổng) và Level Design (sơ đồ chi tiết các khu vực) được thực hiện tỉ mỉ để tạo ra một không gian trải nghiệm logic và hấp dẫn.

Bên cạnh phần nội dung, nhóm cũng dành nhiều tâm huyết cho việc phát triển các cơ chế cốt lõi. Hệ thống điều khiển nhân vật được tối ưu hóa qua bộ công cụ Advanced Character Controller, kết hợp chặt chẽ với Hitbox & Collision Logic để đảm bảo tính chính xác trong các tương tác va chạm và chiến đấu. Để duy trì mạch cảm xúc, cơ chế Level Transition (chuyển màn) cùng hệ thống Death System and Retry (nhân vật chết và hồi sinh) được thiết kế mượt mà, giúp giảm thiểu sự ngắt quãng khi người chơi thất bại.

Trải nghiệm người dùng cũng là một ưu tiên hàng đầu vì thế thông qua Music & Sound Effects, từ âm thanh giao diện đến tiếng động riêng biệt của nhân vật đều được tái hiện một cách chân thực qua đó tăng cảm giác trải nghiệm game. Về mặt thị giác, các giao diện quan trọng như Home (màn hình bắt đầu), User Interface (trong game), End (kết thúc) và tính năng Leaderboard (xếp hạng cạnh tranh) đều được chăm chút kỹ lưỡng.

Cuối cùng, để đảm bảo tính ổn định trên nhiều thiết bị, nhóm đã giải quyết triệt để bài toán Fullscreen và Scaling nhằm tương thích với đa dạng kích thước màn hình. Nhìn chung, các vấn đề mà dự án gặp phải không chỉ nằm ở việc hoàn thành từng chức năng riêng lẻ mà còn ở việc kết nối tất cả thành một hệ thống đồng bộ, mang lại trải nghiệm mượt mà và trọn vẹn nhất cho người chơi.

1.2 Phân tích Chức năng và Yêu cầu Kỹ thuật

1.2.1. Phân tích Chức năng Nhân vật.

Đây là những tính năng trực tiếp mà người chơi có thể thực hiện hoặc hệ thống phải phản hồi:

- Hệ thống điều khiển nhân vật: Hệ thống điều khiển nhân vật phải phản hồi các thao tác di chuyển sang trái (A), sang phải (D), nhảy (Space) và tấn công (Enter).
- Cơ chế chiến đấu và Sinh tồn: Tính toán sát thương dựa trên va chạm giữa vũ khí và kẻ địch. Hệ thống hồi máu thông qua việc nhặt vật phẩm (Health Potion) rơi ra từ quái vật.
- Cơ chế đuổi của Quái : Quái vật cần có hành vi tuần tra, truy đuổi và tấn công người chơi trong phạm vi nhất định.
- Hệ thống quản lý màn chơi: Hệ thống màn chơi phải cho phép chuyển map khi người chơi đi đến rìa bản đồ và thỏa mãn điều kiện tiêu diệt hết kẻ địch trong màn.
- Giao diện tương tác: Trò chơi cần có các màn hình chức năng như StartScreen,PauseOverlay,DeathScreen, MessageOverlay và TheEndScreen để hỗ trợ quá trình tương tác.

1.2.2 Chức năng của orc.

Trong thế giới của "The Whisper of Emerald Forest", Orc không chỉ là những kẻ thù thông thường mà còn là chướng ngại vật chính ngăn cản bước tiến của người chơi. Chúng được thiết kế để tạo ra thử thách về mặt kỹ năng và chiến thuật điều khiển. Các chức năng chính của Orc bao gồm:

Tương tác chiến đấu: Orc sở hữu hệ thống va chạm (Hitbox) riêng biệt, cho phép chúng nhận sát thương từ các đòn tấn công của người chơi. Đồng thời, chúng có khả năng gây sát thương ngược lại nếu người chơi để nhân vật tiếp xúc quá gần hoặc rơi vào tầm tấn công của chúng.

Cơ chế truy đuổi cơ bản: Orc được tích hợp khả năng nhận diện mục tiêu. Chúng có thể tự động tuần tra trong một khu vực nhất định hoặc truy đuổi người chơi khi nhân vật tiến vào phạm vi phát hiện. Điều này buộc người chơi phải quan sát kỹ quy luật di chuyển của chúng để đưa ra phương án xử lý phù hợp.

Khả năng thích nghi địa hình: Khác với các vật thể tĩnh, Orc có khả năng tương tác với trọng lực và địa hình bản đồ. Chúng có thể di chuyển trên các bề mặt phẳng và thực hiện các bước nhảy (Jump) để vượt qua các vật cản nhỏ hoặc các thay đổi về độ cao của nền đất, tạo nên sự linh hoạt trong quá trình truy đuổi.

Cơ chế rơi vật phẩm (Loot System): Để hỗ trợ người chơi trong hành trình đầy nguy hiểm, mỗi khi một Orc bị tiêu diệt, chúng có xác suất rơi ra các vật phẩm hỗ trợ như bình hồi máu (Health Potion). Đây là nguồn cung cấp tài nguyên quan trọng giúp người chơi duy trì chỉ số sinh mệnh để tiếp tục khám phá các màn chơi tiếp theo.

Điều kiện tiên quyết của màn chơi: Orc đóng vai trò là "chìa khóa" logic. Để hoàn thành một khu vực và tiến tới cổng dịch chuyển (Teleport), người chơi bắt buộc phải quét sạch toàn bộ số lượng Orc hiện diện trên bản đồ. Chức năng này biến mỗi cuộc đối đầu với Orc thành một nhiệm vụ bắt buộc để mở khóa tiến trình của câu chuyện.

1.2.3 Thách thức Kỹ thuật và Giải pháp Công nghệ.

Xử lý va chạm đa đối tượng: Giải quyết bài toán va chạm giữa Player - Địa hình, Player - Enemy, và Enemy - Địa hình trong cùng một thời điểm.

Đồng bộ hóa Animation: Đảm bảo âm thanh chém kiếm vang lên đúng khoảnh khắc hình ảnh vũ khí chạm vào quái vật.

Quản lý bộ nhớ: Giải phóng các thực thể đã chết hoặc bình máu đã ăn khỏi danh sách ArrayList để tránh tràn bộ nhớ khi chơi lâu.

Xử lý mô phỏng bản đồ: Đôi khi bản đồ bị lật tile, méo hoặc tỉ lệ không tương thích, có những thuật toán riêng để xử lý.

1.2.4 Yêu cầu về dữ liệu và Tài nguyên (Data & Assets Requirements).

Đảm bảo hệ thống quản lý tài nguyên hiệu quả:

- Quản lý Sprite Sheet: Hệ thống phải có khả năng cắt và phát các hoạt ảnh (Animation) từ các tấm ảnh ghép lớn một cách chính xác.

- Xử lý bản đồ (Map Data): Đọc và phân tích dữ liệu từ tệp .tmx để xác định vị trí các vật cản (Collision Block), địa hình và điểm xuất hiện (Spawn points).
- Âm thanh: Hỗ trợ phát nhạc nền vòng lặp (Loop) và các âm thanh hiệu ứng (SFX) đa kênh.

1.2.5 Yêu cầu về giao diện người dùng (UI/UX Requirements).

Giao diện người dùng của trò chơi cần được thiết kế theo hướng trực quan, dễ quan sát và hỗ trợ người chơi nắm bắt trạng thái hệ thống trong suốt quá trình chơi. Ngoài yếu tố thẩm mỹ, UI còn phải góp phần điều hướng hành vi người chơi thông qua các màn hình chức năng và các thành phần hiển thị trong lúc chơi.

- Tính trực quan và Hiển thị trạng thái sinh tồn (Visual HUD).

  - Giao diện người dùng trong trò chơi (HUD) cần được thiết kế theo hướng tối giản nhưng vẫn bảo đảm cung cấp đầy đủ thông tin quan trọng cho người chơi. Mục tiêu của HUD là giúp người chơi theo dõi trạng thái của nhân vật trong thời gian thực mà không làm phân tán sự tập trung khỏi quá trình điều khiển và chiến đấu.
  - Bộ đếm thời gian (Timer) : Timer được hiển thị bên trên, góc phải màn hình nhằm theo dõi tổng thời gian hoàn thành lượt chơi của người chơi. Thông tin này không chỉ giúp người chơi nhận biết tiến độ hiện tại mà còn tạo thêm áp lực và động lực trong quá trình vượt màn. Trong hệ thống hiện tại, thời gian được cộng dồn liên tục khi nhân vật đang trong trạng thái chơi và được cập nhật theo thời gian thực lên giao diện. Điều này đặc biệt quan trọng vì thành tích của người chơi trên leaderboard được xác định dựa trên thời gian hoàn thành, tức là thời gian càng thấp thì thứ hạng càng cao.
  - Thanh máu : Bên cạnh bộ đếm thời gian, thanh máu cũng là một thành phần quan trọng của HUD. Thanh máu là thành phần quan trọng nhất trong hệ thống hiển thị trạng thái sinh tồn. Nó thể hiện trực tiếp mức độ an toàn của nhân vật tại mọi thời điểm. Trong hệ thống hiện tại, lượng HP của nhân vật được quản lý trong đối tượng người chơi và được cập nhật mỗi khi nhân vật chịu sát thương hoặc được hồi phục. Khi nhân vật bị quái tấn công, hàm takeDamage() được gọi để giảm giá trị HP. Sau đó, thanh máu trên HUD sẽ thay đổi tương ứng với lượng máu còn lại, từ đó tạo ra phản hồi trực quan ngay lập tức cho người chơi. Cơ chế này giúp

người chơi dễ dàng nhận biết khi nào nhân vật đang ở trạng thái nguy hiểm.

Ngoài ra, cơ chế hiển thị vật phẩm hồi phục cũng đóng vai trò hỗ trợ quan trọng. Khi quái bị tiêu diệt và rơi ra Potion, vật phẩm này cần được đặt ở vị trí dễ nhận biết và có hình ảnh đủ nổi bật so với môi trường xung quanh. Điều này giúp người chơi nhanh chóng xác định được vị trí của vật phẩm để tiếp cận và sử dụng khi cần thiết. Nhờ đó, hệ thống HUD và các yếu tố hiển thị liên quan không chỉ đơn thuần cung cấp thông tin, mà còn trực tiếp hỗ trợ trải nghiệm chiến đấu, sinh tồn và điều hướng trong trò chơi.

- Hệ thống Thông báo và Chỉ dẫn ngữ cảnh (Contextual Feedback).

  - Thông báo quy tắc: Khi người chơi cố gắng thực hiện một hành động chưa đủ điều kiện (ví dụ: bước vào cổng dịch chuyển khi chưa tiêu diệt hết kẻ thù), một thông báo rõ ràng như "You have to slay all the enemy" phải xuất hiện ngay lập tức tại vị trí trung tâm hoặc thông qua lớp phủ MessageOverlay.

  - Ngoài việc thông báo lỗi thao tác, các màn hình như `PauseOverlay`, `DeathScreen` và `TheEndScreen` cũng đóng vai trò phản hồi trạng thái của trò chơi. Mỗi màn hình đều được thiết kế để truyền đạt rõ ràng trạng thái hiện tại và cho phép người chơi tiếp tục, chơi lại hoặc quay về menu chính.

1.2.6 Quản lý Độ phân giải và Tỷ lệ hiển thị (Screen Scaling & Aspect Ratio).

Trong trò chơi, hệ thống hiển thị được xây dựng dựa trên một độ phân giải ảo cố định là `800 x 480`, tương ứng với không gian vẽ chuẩn của gameplay và giao diện. Toàn bộ nội dung như bản đồ, nhân vật, HUD và các màn hình chức năng đều được dựng trong hệ tọa độ này trước khi được phóng to lên kích thước cửa sổ thực tế. Cách tiếp cận này giúp trò chơi duy trì được bố cục ổn định trên nhiều độ phân giải khác nhau.

Hình ảnh 1.1: Các option chỉnh kích thước màn hình.

Trò chơi hỗ trợ các chế độ hiển thị gồm `800x480`, `1200x720`, `1600x960` và `Full Screen`. Việc chuyển đổi giữa các chế độ này được thực hiện thông qua menu `Resolution`. Khi người dùng thay đổi kích thước cửa sổ hoặc chuyển sang toàn màn hình, hệ thống sẽ tính lại hệ số scale dựa trên kích thước hiện tại của cửa sổ và độ phân giải ảo ban đầu.

Để tránh hiện tượng méo hình, game sử dụng cơ chế bảo toàn tỷ lệ khung hình bằng cách lấy hệ số scale nhỏ hơn giữa chiều rộng và chiều cao. Phần không gian còn thừa được chừa lại dưới dạng viền đen, nhờ đó nội dung không bị kéo giãn theo chiều ngang hoặc chiều dọc. Cách làm này giúp các thành phần đồ họa và giao diện luôn giữ được bố cục nhất quán trong mọi chế độ hiển thị đang được hỗ trợ.

1.2.7 Hệ thống Menu và Luồng chuyển cảnh (Navigation & Transition).

Trải nghiệm người dùng không chỉ nằm ở trong màn chơi mà còn ở cách họ tương tác với hệ thống quản lý game:

Menu đa tầng: Hệ thống menu chính (StartScreen), menu tạm dừng (PauseOverlay) và màn hình kết thúc (TheEndScreen) phải có cấu trúc nhất quán. Các nút bấm cần có hiệu ứng phản hồi hoặc khi nhấn xác nhận (nút Enter) để xác nhận hành động của người chơi.

Luồng xử lý thua cuộc (Retry Logic): Khi nhân vật chết, giao diện DeathScreen phải cung cấp lựa chọn "Thử lại" (Retry) hoặc "Thoát" (M.Menu) một cách nhanh

chóng, giúp người chơi quay lại nhịp độ game mà không phải trải qua các bước tải dữ liệu rườm rà.

1.3 Giao diện và quy tắc trò chơi

1.3.1 Thiết kế Giao diện người dùng (User Interface Design).

Giao diện của "Whisper of the Emerald Forest" được thiết kế theo phong cách tối giản nhưng vẫn đảm bảo cung cấp đầy đủ thông tin cần thiết cho người chơi mà không gây xao nhãng khỏi trải nghiệm hành động.

Giao diện trong màn chơi (HUD - Heads-up Display):

Thanh trạng thái sinh mạng (Health Bar): Được đặt cố định ở góc trên bên trái màn hình. Thanh máu sử dụng màu đỏ đặc trưng để biểu thị lượng HP hiện có của nhân vật. Khi nhân vật chịu sát thương, thanh máu sẽ giảm dần theo thời gian thực (Real-time).

Hệ thống thông báo ngữ cảnh: Khi người chơi tiếp cận các khu vực đặc biệt hoặc chưa đủ điều kiện qua màn, bảng thông báo (Overlays) như "You have to slay all the enemies" sẽ xuất hiện tại trung tâm màn hình để điều hướng người chơi.

Hiệu ứng phản hồi thị giác: Khi quái vật hoặc nhân vật chính bị trúng đòn, hệ thống sẽ hiển thị hoạt ảnh trúng đòn và phát âm thanh (Grunt) để xác nhận va chạm đã xảy ra thành công.

Màn hình trạng thái của trang chủ (Game Screens):

Hình ảnh 1.2: Giao diện chính .

Giao diện người dùng của trò chơi được phân tách thành các màn hình chức năng riêng biệt, giúp người chơi dễ dàng điều hướng và nắm bắt thông tin. Dưới đây là các trạng thái màn hình chính được thiết lập trong Game:

Màn hình Menu chính (Main Menu): Đây là giao diện đầu tiên khi khởi động trò chơi, được thiết kế với hình ảnh trung tâm là nhân vật Hiệp sĩ và bối cảnh rừng Emerald huyền bí. Danh sách các tùy chọn bao gồm:

- PLAY: Kích hoạt trạng thái bắt đầu màn chơi đầu tiên.
- RESOLUTION: Cho phép người chơi tùy chỉnh kích thước hiển thị (Cửa sổ hoặc Toàn màn hình) để phù hợp với thiết bị.
- INTRO: Giới thiệu ngắn gọn về cốt truyện hoặc bối cảnh của "Emerald Forest".
- GUIDE: Cung cấp các hướng dẫn về phím điều khiển (di chuyển, tấn công, tương tác).
- LEADERBOARD : Hệ thống xếp hạng cho từng vòng chơi của người chơi.
- CREDITS: Hiển thị thông tin đội ngũ phát triển (The Nokora's Team).
- EXIT: Thoát khỏi trò chơi và đóng ứng dụng hoàn toàn.

Hệ thống điều khiển nhanh tại Menu:

- Phím ENTER: Dùng để xác nhận lựa chọn (ENTER to select).
- Phím ESC: Dùng để thoát nhanh ra khỏi ứng dụng (ESC to quit).

1.3.2 Quy tắc trò chơi cốt lõi (Core Game Rules).

Quy tắc trò chơi là tập hợp các logic toán học và vật lý được lập trình để tạo ra tính thử thách và sự công bằng.

Các phím điều khiển nhân vật :Nhân vật hiệp sĩ được điều khiển thông qua sự kết hợp giữa bàn phím và chuột, cho phép thực hiện các tổ hợp hành động nhanh:

- Di chuyển (Movement):
  - Phím A: Di chuyển nhân vật sang bên trái.

- Phím D: Di chuyển nhân vật sang bên phải.
- Phím Space : Cho phép nhân vật nhảy lên để vượt địa hình hoặc né đòn.

- Tấn công (Enter): Thực hiện đòn chém bằng kiếm.
- Cơ chế Combo: Nếu người chơi nhấn enter liên tiếp, nhân vật sẽ thực hiện chuỗi hoạt ảnh tấn công gồm 3 giai đoạn với sát thương và phạm vi quét kiếm khác nhau.

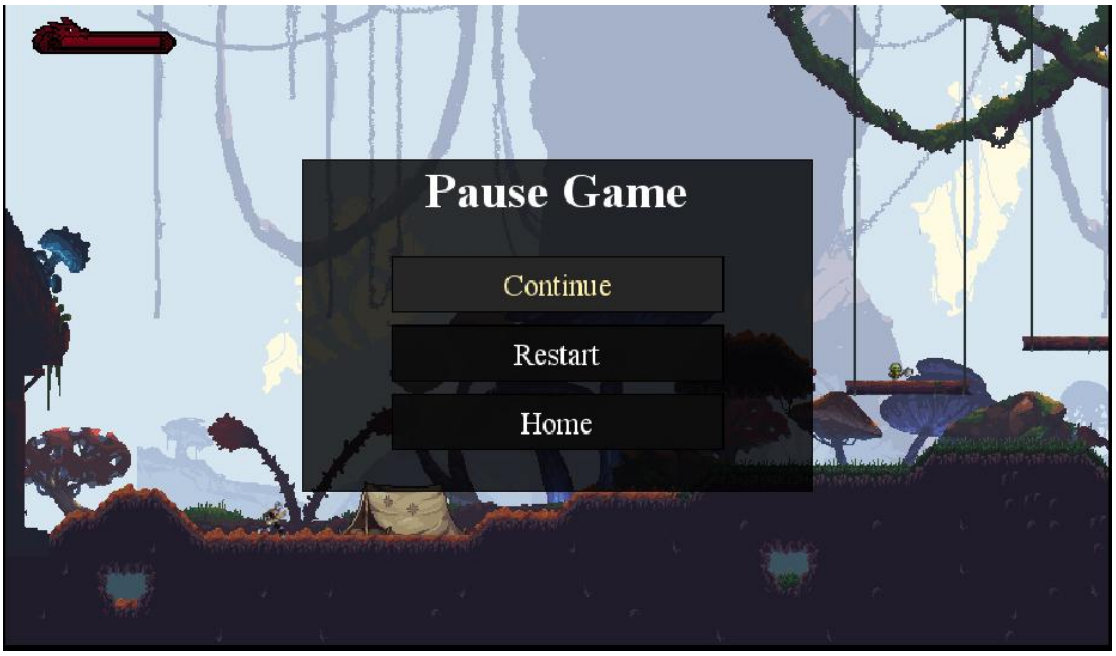
Quy tắc di chuyển và Vật lý địa hình:

- Trọng lực (Gravity): Nhân vật luôn chịu tác động của lực hút ảo, khiến nhân vật rơi xuống cho đến khi va chạm với các thực thể cứng (Collision Block).
- Giới hạn không gian: Nhân vật không thể di chuyển ra khỏi phạm vi của bản đồ được định nghĩa trong file .tmx. Các vùng biên sẽ đóng vai trò như những bức tường vô hình ngăn chặn việc rơi ra ngoài không gian game hoặc cũng có thể là cổng dịch chuyển đưa nhân vật đến map khác dựa trên điều kiện đã thỏa mãn.

Quy tắc chiến đấu và Tính toán sát thương: Khi người chơi thực hiện đòn đánh, hệ thống tạo ra một vùng va chạm tạm thời tương ứng với chuyển động của vũ khí. Nếu vùng này giao nhau với vùng va chạm của quái trong khoảng thời gian hợp lệ của animation, quái sẽ nhận sát thương. Ở chiều ngược lại, quái cũng có vùng tấn công riêng để gây sát thương cho người chơi khi ở đủ gần. Máu của nhân vật sẽ giảm thông qua hàm `takeDamage()`, còn việc hồi phục được xử lý bằng `heal()` khi nhặt được bình máu.

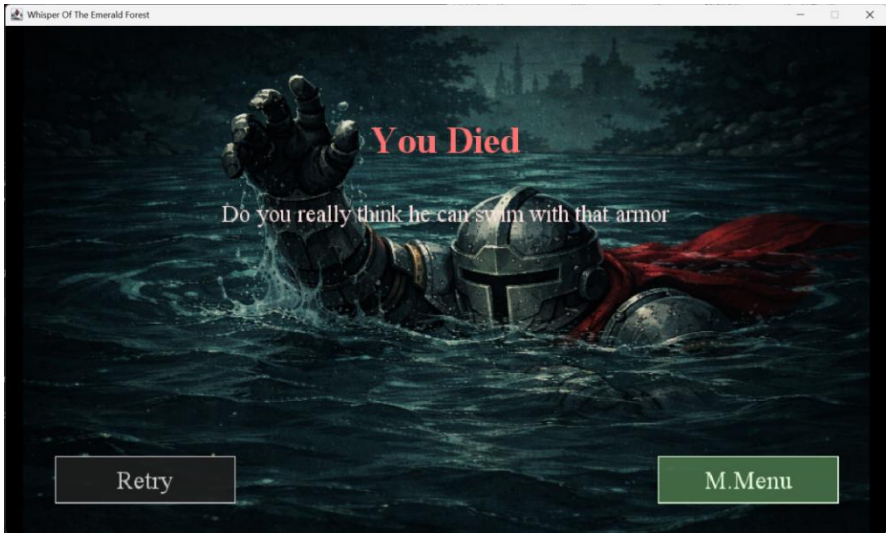
Quy tắc tiến trình và Qua màn (Level Progression): Trò chơi sử dụng cơ chế qua màn theo điều kiện tiêu diệt hết kẻ địch. Người chơi chỉ có thể rời khỏi bản đồ khi đi tới rìa màn và danh sách quái vật trong màn đã được xóa hết. Nếu vẫn còn quái, hệ thống sẽ chặn di chuyển và hiển thị thông báo yêu cầu người chơi tiếp tục chiến đấu. Sau khi hoàn thành điều kiện này, người chơi mới có thể chuyển sang màn kế tiếp hoặc tiến tới phần kết.

Tạm dừng (Pause Logic): Khi đang chơi, người chơi có thể mở menu tạm dừng để dừng quá trình cập nhật gameplay. Ở trạng thái này, các logic điều khiển nhân vật, quái vật và chuyển động trong màn sẽ không tiếp tục xử lý cho đến khi người chơi quay lại game hoặc chọn hành động khác như chơi lại hoặc quay về menu chính. Cơ chế này giúp quá trình tạm dừng diễn ra đơn giản nhưng hiệu quả.

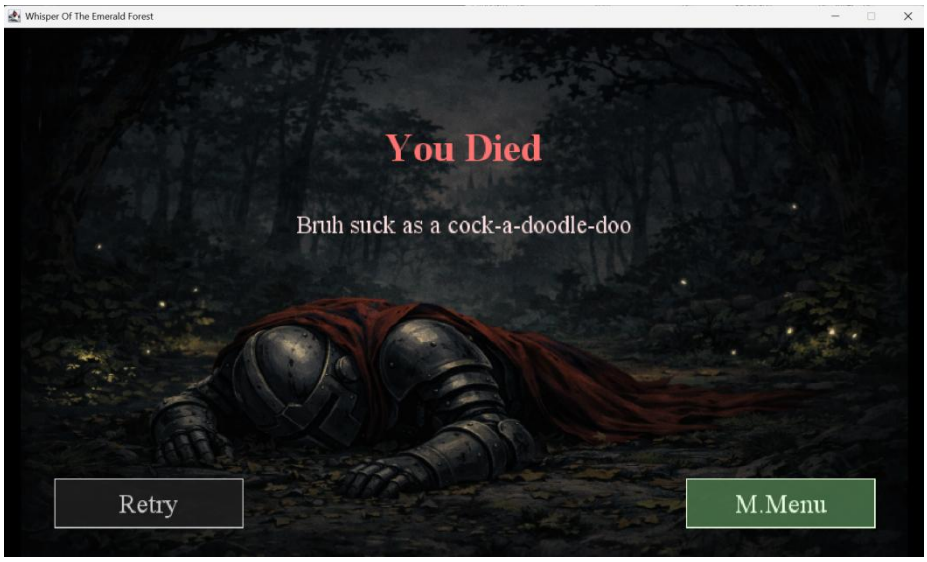


Hình ảnh 1.3: Trạng thái dừng game.

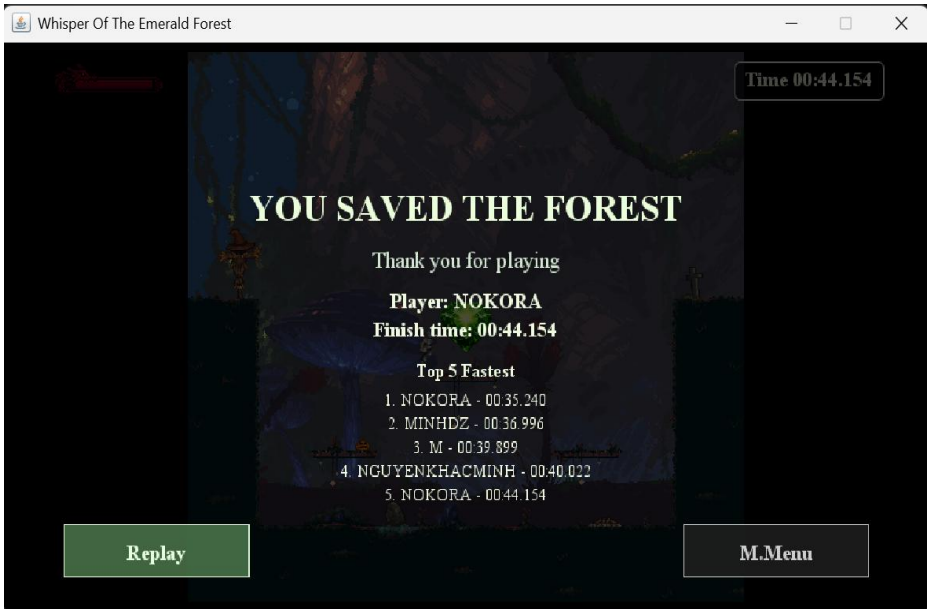
Kết thúc màn chơi (Game Flow Rules): Khi nhân vật hết máu do bị quái đánh chết hoặc chết do môi trường như rơi xuống nước (va chạm với water block), trò chơi sẽ chuyển sang `DeathScreen`. Tại đây, người chơi có thể chọn chơi lại màn hiện tại hoặc trở về menu chính. Ngược lại, khi hoàn thành màn cuối và chạm vào viên Emerald, game sẽ chuyển sang trạng thái chiến thắng, phát âm thanh tương ứng và hiển thị `TheEndScreen`, nơi người chơi có thể xem thời gian hoàn thành, preview leaderboard và lựa chọn chơi lại hoặc quay về menu.



Hình ảnh 1.4 : Chết do đuối nước.



Hình ảnh 1.5: Chết do quái đánh.



Hình ảnh 1.6 : Kết thúc và thành tích.

Trường hợp Hoàn thành (Victory): Sau khi nhặt được viên ngọc, trò chơi sẽ chuyển sang màn hình Credits để cảm ơn người chơi, hiển thị kết quả chạy và top 5 người chơi nhanh nhất trước khi chơi lại để speed run tựa game hoặc trở về màn hình bắt đầu.

1.3.3 Quy tắc quản lý Tài nguyên và Vật phẩm.

Cơ chế hồi phục và giới hạn chỉ số (Potion System): Khi tiêu diệt quái vật Orc, hệ thống sẽ tính toán xác suất để tạo ra vật phẩm hồi máu (Potion) ngay tại vị trí rơi. Người chơi khi nhặt vật phẩm sẽ được hồi phục từ 1 đến 2 đơn vị HP tùy theo tỷ lệ

ngẫu nhiên. Để đảm bảo tính cân bằng, lượng máu sau khi hồi phục được kiểm soát nghiêm ngặt bằng hàm ràng buộc, đảm bảo không vượt quá giới hạn sinh mệnh tối đa của nhân vật (7 HP).

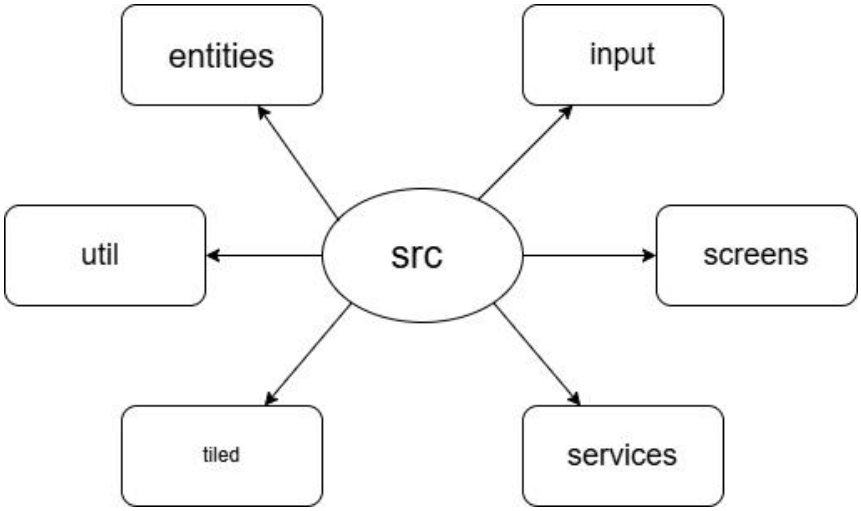
Quản lý âm thanh và nhạc nền theo ngữ cảnh: Hệ thống âm thanh được vận hành linh hoạt dựa trên trạng thái thực tế của trò chơi. Nhạc nền bối cảnh (LoopingSound) sẽ phát lặp lại xuyên suốt quá trình khám phá màn chơi và chỉ thay đổi khi người chơi chuyển dịch dữ liệu sang bản đồ mới. Bên cạnh đó, các hiệu ứng âm thanh tức thời (OneShotSound) và nhạc trạng thái sẽ được kích hoạt riêng biệt khi người chơi tương tác với vật phẩm hoặc khi đạt đến các kết quả thắng/thua (Victory/Death Screen).

CHƯƠNG 2 KIẾN TRÚC HỆ THÔNG GAME

Trong quá trình phát triển trò chơi "Whisper of the Emerald Forest", việc tách biệt giữa mã nguồn xử lý logic và dữ liệu tài nguyên tĩnh là một nguyên tắc thiết kế quan trọng, trong đó hệ thống tài nguyên (Assets) không chỉ đóng vai trò là phần nhìn và phần nghe mà còn là cơ sở dữ liệu cấu trúc nên các màn chơi phức tạp, còn mã nguồn (Source Code - SRC) đóng vai trò là "bộ não" điều khiển và kết nối các thành phần đó thành một thể thống nhất. Thư mục mã nguồn được tổ chức chặt chẽ theo các gói chức năng (packages) như ***entities*** để định nghĩa hành vi nhân vật, ***screens*** để quản lý luồng giao diện, và ***tiled*** để giải mã dữ liệu bản đồ, giúp biến những hình ảnh và âm thanh tĩnh trong ***Assets*** thành các thực thể có khả năng tương tác, di chuyển và phản hồi theo ý muốn của người chơi. Sự phối hợp nhịp nhàng này được thể hiện qua việc các lớp tiện ích trong ***util*** nạp tài nguyên từ bộ nhớ, sau đó các thuật toán xử lý va chạm và logic trò chơi trong SRC sẽ tính toán và hiển thị chúng lên màn hình thông qua hệ thống Camera, tạo nên một quy trình vận hành khép kín, tối ưu hóa hiệu suất và giúp việc mở rộng nội dung game trở nên linh hoạt, dễ dàng bảo trì hơn.

2.1 Sơ đồ tổ chức thư mục (Project Directory Structure)

Hệ thống code game: Thư mục SRC (Source Code) đóng vai trò là "bộ não" điều khiển toàn bộ các hoạt động của trò chơi "Whisper of the Emerald Forest". Đây là nơi chứa đựng các tập tin mã nguồn Java, định nghĩa toàn bộ quy luật vận hành, trí tuệ nhân tạo của quái vật, cơ chế vật lý và cách thức mà người chơi tương tác với thế giới ảo. Nếu các tệp tin trong Assets là phần xác (hình ảnh, âm thanh) thì SRC chính là phần hồn, giúp kết nối các tài nguyên tĩnh đó lại để tạo thành một hệ thống trò chơi hoàn chỉnh và có khả năng phản hồi.



Hình ảnh 2.1: Mô tả cấu trúc code game.

Cấu trúc chương trình:

- Package entties :

- Class HealthPotion :Lưu trữ tọa độ vật lý và logic hồi phục sinh mệnh cho thực thể bình máu.
- Class OrcEnemy : lưu trữ thông tin chung của quái vật
- Class Player :Lưu trữ thông tin của nhân vật

-Package input :

- Class InputState :xử lý các sự kiện đầu vào từ bàn phím để điều khiển hành động của nhân vật.

-Package screens :

- Class DeathScreen :Xử lý hiển thị thông báo tử vong và lựa chọn chơi lại.
- Class GameScreen : Xử lý logic và hiển thị màn chơi chính cấp độ 1.
- Class GameScreen2 :Xử lý logic và hiển thị màn chơi chính cấp độ 2.
- Class MessageOverlay : Xử lý hiển thị các khung hội thoại và văn bản hướng dẫn.
- Class PauseOverlay : Xử lý menu tạm dừng và các tùy chọn tiếp tục trò chơi.
- Class StartScreen :Xử lý menu khởi động, cấu hình và điều hướng ban đầu.
- Class TheEndScreen :Xử lý hiển thị kết quả và thông báo hoàn thành trò chơi.

-Package serviecs :

- Class LeaderboardEntry :Xử lý lưu trữ thông tin tên người chơi và điểm số đạt được.
- Class LeaderboardService :Xử lý logic đọc, ghi và sắp xếp danh sách điểm cao.

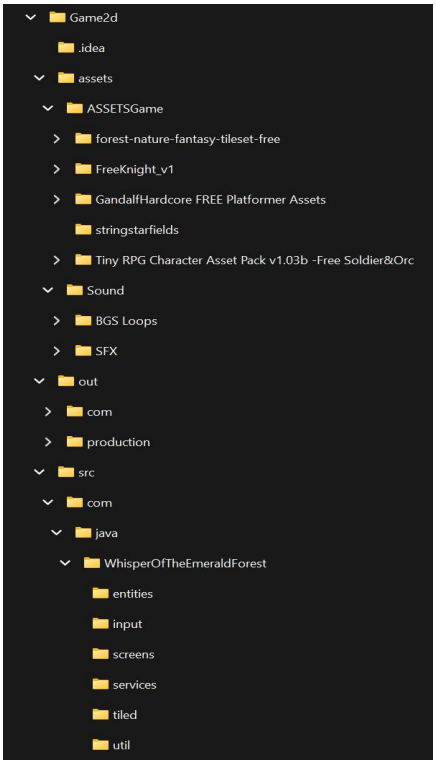
-Package tiled :

- Class TiledLayer :Lưu trữ dữ liệu các tầng (lớp) gạch và thuộc tính hiển thị tương ứng.
- Class TiledMap :Xử lý cấu trúc dữ liệu tổng thể của bản đồ, bao gồm tầng gạch và bộ tài nguyên.
- Class TiledMapRenderer :Xử lý việc vẽ các ô gạch lên màn hình và tối ưu hóa khung nhìn (Culling).
- Class Tileset :Xử lý quản lý tập hợp các hình ảnh ô gạch và các chỉ số định danh (GID).
- Class TmxMapLoader :Xử lý việc đọc và phân tích tệp dữ liệu XML (.tmx) từ phần mềm thiết kế bản đồ.

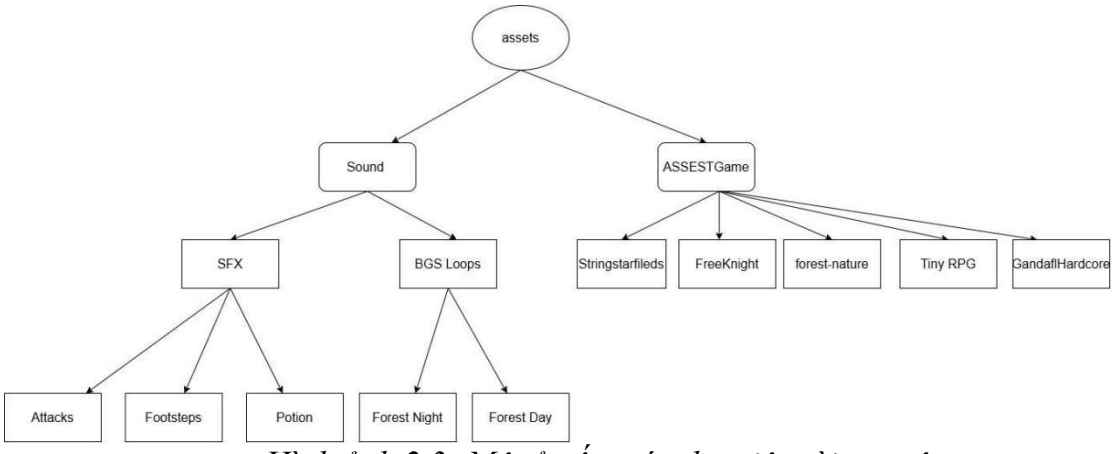
- Package util :

- Class Animation :quản lý khung hình và thời gian hiển thị hoạt ảnh chuyển động.
- Class Assets :lưu trữ bộ nhớ đệm (Cache) các tài nguyên hình ảnh.
- Class Camera2D :Xử lý quản lý khung nhìn và chuyển đổi hệ tọa độ thế giới sang màn hình.
- Class GraphicsUtil :Xử lý các phép biến đổi hình học và hiệu ứng lật hình khi vẽ đồ họa.
- Class LoopingSound :Xử lý phát và quản lý các luồng âm thanh lặp lại liên tục.
- Class OneShotSound :Xử lý phát các hiệu ứng âm thanh tức thời và tự động giải phóng bộ nhớ.
- Class TextUtil :Xử lý tính toán độ dài và tự động ngắt dòng văn bản theo khung hình.
- Class Core :Điều phối vòng đời trò chơi và chuyển đổi giữa các màn hình chức năng.
- Class GamePanel :Xử lý luồng trò chơi chính (Thread) và quản lý tốc độ khung hình
- Class Main : Xử lý khởi tạo cửa sổ ứng dụng và bắt đầu chương trình.

Hệ thông tài nguyên: Hệ thống Assets đóng vai trò là kho lưu trữ tài nguyên tĩnh, cung cấp "diện mạo" và "âm thanh" để hiện thực hóa thế giới đồ họa của trò chơi. Được tổ chức tách biệt hoàn toàn với mã nguồn logic, hệ thống này bao gồm các thư mục chuyên biệt như ASSETSGame để quản lý hình ảnh nhân vật, quái vật và các Sprite Sheet hệ thống; thư mục Maps chứa dữ liệu địa hình .tmx được thiết kế từ công cụ Tiled; và thư mục Sound lưu trữ nhạc nền cùng hiệu ứng âm thanh. Việc quản lý tập trung thông qua lớp tiện ích Assets.java giúp tối ưu hóa bộ nhớ bằng cách chỉ nạp tài nguyên một lần duy nhất nhưng cho phép nhiều thành phần khác nhau trong game cùng truy xuất và sử dụng. Chính sự phân tách rạch ròi này đã tạo nên một cơ sở dữ liệu tài nguyên ổn định, giúp lập trình viên dễ dàng thay đổi phong cách nghệ thuật hoặc nâng cấp chất lượng hình ảnh mà không gây ảnh hưởng đến cấu trúc vận hành cốt lõi của hệ thống.



Hình ảnh 2.2: Thư mục tài nguyên.



Hình ảnh 2.3: Mô tả cấu trúc thư viện tài nguyên.

Chức năng :

- ASSETSGame/: Chứa hình ảnh nhân vật, quái vật và các Sprite Sheet hệ thống.
    - FreeKnight/: Chứa các bộ Sprite Sheet (chuỗi hình ảnh hoạt họa) cho nhân vật chính. Các tệp này được mã nguồn sử dụng để cắt ghép và tạo hiệu ứng chuyển động như tấn công (Attack), di chuyển (Move) hay nhảy (Jump).
    - springstarfields/, forest-nature-…./, …: Lưu trữ các thành phần đồ họa môi trường, cây cối và phông nền giúp tạo nên không gian đặc trưng của khu rừng Emerald.
    - Dragonhpbar.png: Tệp đồ họa giao diện (UI) dùng để hiển thị thanh trạng thái sinh mệnh của nhân vật.
- Thư mục Sound/ (Âm thanh bối cảnh và Hiệu ứng):
    - BGS LOOPS/: Chứa các bản nhạc nền (Background Music) có khả năng phát lặp lại vô hạn, thay đổi tùy theo từng màn chơi (như Forest Day hay Forest Night).
    - SFX/ (Sound Effects): Chứa các tệp âm thanh ngắn phản hồi cho hành động của người chơi như: Attacks (tiếng vũ khí khi người chơi thực hiện đòn đánh), Footsteps (hiệu ứng tiếng bước chân thay đổi theo bề mặt di chuyển), Grunt (âm thanh phản hồi khi nhân vật hoặc quái vật chịu sát thương), Potion (hiệu ứng thính giác đặc trưng khi người chơi nhặt vật phẩm hồi phục)

2.2 Mối liên kết giữa các thành phần (System Interconnections)

Sự vận hành ổn định của "Whisper of the Emerald Forest" dựa trên các mối liên kết logic chặt chẽ giữa các thư mục chức năng:

2.2.1. Liên kết giữa Tiled, Assets và Screens.

Thư mục tiled đóng vai cầu nối trung gian. Khi GameScreen (trong screens) được khởi tạo, nó sử dụng TmxMapLoader để truy cập vào đường dẫn tệp trong assets/Maps. Quá trình này kết hợp với lớp Assets (trong util) để nạp các hình ảnh Tileset tương ứng. Kết quả cuối cùng là TiledMapRenderer sẽ chuyển đổi dữ liệu thô này thành hình ảnh địa hình hiển thị trên màn hình dựa trên tọa độ của Camera.

2.2.2. Liên kết giữa Entities và Tiled (Hệ thống va chạm).

Đây là mối liên kết sống còn của trò chơi. Các thực thể trong entities khi thực hiện di chuyển sẽ liên tục truy vấn dữ liệu từ TiledLayer (trong tiled) để lấy mã định danh (Gid) tại tọa độ tương ứng. Nếu Gid đó đại diện cho một ô gạch vật chất (như tường, đất), logic trong entities sẽ ngăn chặn việc thay đổi tọa độ, tạo nên hiệu ứng va chạm vật lý.

2.2.3. Liên kết giữa Services và Screens (Tính năng trực tuyến).

Gói services cung cấp khả năng mở rộng cho giao diện. Lớp LeaderboardService thực hiện các yêu cầu mạng không đồng bộ để gửi hoặc lấy dữ liệu từ Supabase. Thông tin này sau đó được ánh xạ vào LeaderboardEntry và chuyển lại cho các lớp trong screens để hiển thị bảng thành tích cho người chơi mà không làm treo luồng xử lý chính của game.

2.2.4. Liên kết giữa Util và Hệ thống điều khiển.

Thư mục util cung cấp các dịch vụ nền tảng:

Camera2D: Kết nối vị trí của Player (trong entities) với khung hình hiển thị của screens, đảm bảo góc nhìn luôn đi theo nhân vật chính.

Assets: Quản lý bộ nhớ tập trung, đảm bảo hình ảnh chỉ được nạp một lần nhưng có thể sử dụng bởi nhiều đối tượng khác nhau trong hệ thống.

2.3 Đánh giá tính hiệu quả của cấu trúc

Việc tích hợp thêm gói services vào mô hình phân lớp hiện tại giúp dự án đạt được tính Encapsulation (Đóng gói) và Scalability (Khả năng mở rộng) cao. Người

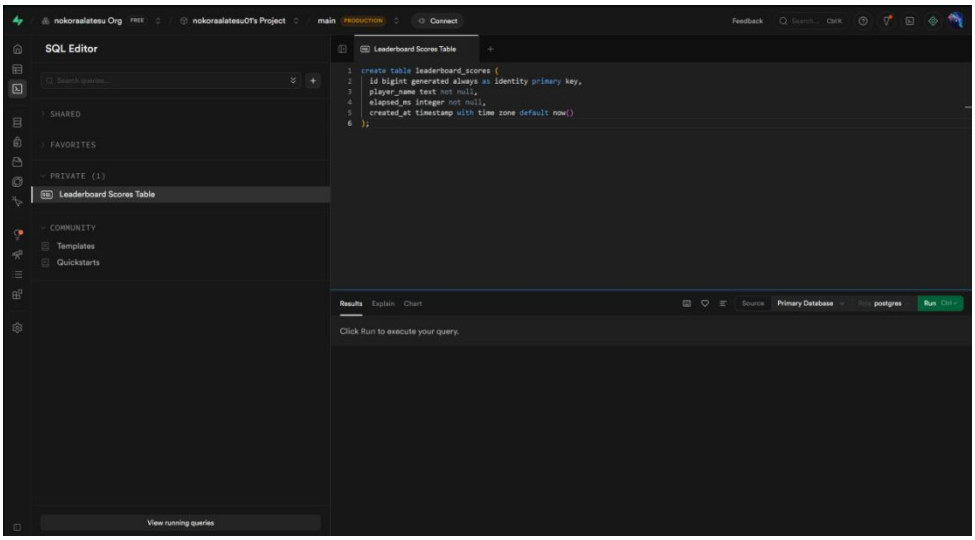
phát triển có thể thay đổi nhà cung cấp dịch vụ lưu trữ trực tuyến trong services hoặc nâng cấp thuật toán AI trong entities mà không làm ảnh hưởng đến cấu trúc nạp bản đồ hay hiển thị giao diện. Cách tổ chức này giúp giảm thiểu lỗi hệ thống và tối ưu hóa quy trình bảo trì dự án lâu dài.

2.4 Kết nối cơ sở dữ liệu

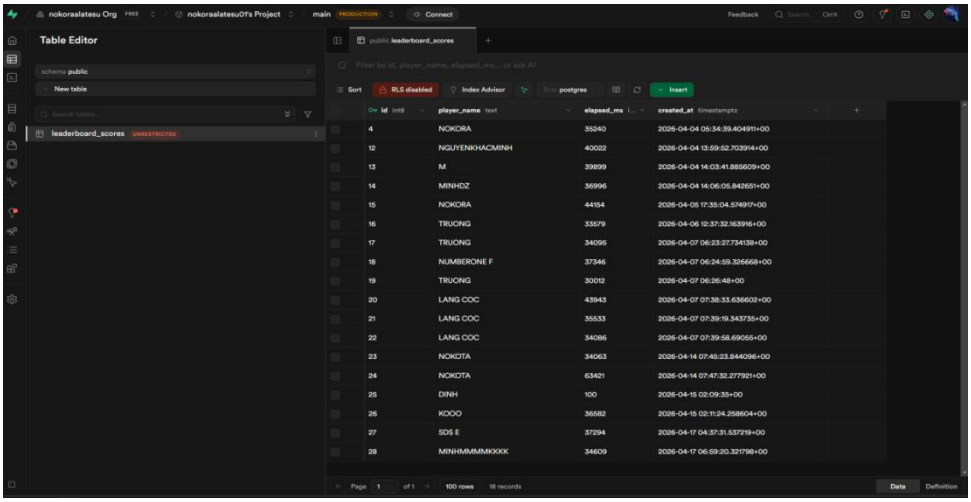
Trong dự án này, quá trình kết nối cơ sở dữ liệu được thực hiện thông qua dịch vụ Supabase theo mô hình gián tiếp, tức là giao tiếp với Supabase thông qua REST API. Ở lớp Core, chương trình cấu hình sẵn địa chỉ dự án Supabase và khóa truy cập API, sau đó truyền hai thông tin này vào lớp LeaderboardService khi khởi tạo hệ thống. Đây là bước thiết lập nền tảng để toàn bộ chức năng liên quan đến bảng xếp hạng có thể truy cập đúng dịch vụ từ xa và xác thực hợp lệ với máy chủ.

Sau khi hoàn tất khởi tạo, LeaderboardService đóng vai trò là tầng trung gian chịu trách nhiệm trao đổi dữ liệu giữa trò chơi và Supabase. Khi cần tải bảng xếp hạng, dịch vụ này gửi một yêu cầu HTTP đến endpoint REST tương ứng với bảng leaderboard_scores trên Supabase. Yêu cầu này kèm theo các thông tin xác thực bằng API key, đồng thời chỉ định rõ các trường dữ liệu cần lấy như tên người chơi, thời gian hoàn thành và thời điểm tạo bản ghi. Dữ liệu phản hồi từ Supabase được trả về dưới dạng JSON, sau đó được phân tích và chuyển thành danh sách các đối tượng LeaderboardEntry trong bộ nhớ của chương trình. Nhờ đó, các màn hình giao diện không cần xử lý trực tiếp kết nối mạng mà chỉ đọc dữ liệu đã được nạp sẵn từ bộ nhớ đệm.

Ở chiều ngược lại, khi người chơi hoàn thành trò chơi, hệ thống sẽ gửi thành tích mới lên Supabase. Quá trình này bắt đầu từ màn hình kết thúc, nơi trò chơi gọi chức năng nộp điểm hiện tại. Từ đó, Core chuyển yêu cầu sang LeaderboardService, và lớp này thực hiện một yêu cầu HTTP POST để gửi dữ liệu gồm tên người chơi và thời gian hoàn thành lên bảng xếp hạng trên Supabase. Sau khi máy chủ phản hồi thành công, dịch vụ tiếp tục tải lại dữ liệu bảng xếp hạng mới nhất để cập nhật bộ nhớ đệm, bảo đảm rằng thông tin hiển thị trên giao diện luôn đồng bộ với dữ liệu đã được lưu trên hệ thống từ xa.



Hình ảnh 2.4: Code SQL .



Hình ảnh 2.5 : Dữ liệu và thành tích người chơi

CHƯƠNG 3. CHI TIẾT CÀI ĐẶT VÀ CẤU TRÚC HỆ THỐNG

3.1 Phần code nhân vật

3.1.1. Thực thể Người chơi (Player)

Lớp Player được thiết kế theo mô hình máy trạng thái hữu hạn , đóng vai trò là thực thể phức tạp nhất trong hệ thống. Nó không chỉ quản lý hình ảnh hiển thị mà còn tích hợp các logic về vật lý, điều khiển đầu vào (input) và hệ thống chiến đấu đa tầng.

Quản lý trạng thái và Hoạt ảnh (State & Animation Management):

```
public class Player {
  public enum State {
    IDLE,
    RUN,
    JUMP,
    FALL,
    HURT,
    ATTACK1,
    ATTACK2,
    ATTACK3,
    DEATH
  }
```

Để đảm bảo sự mượt mà trong chuyển động, người chơi được quản lý thông qua một danh sách các trạng thái (enum State) bao gồm: IDLE, RUN, JUMP, FALL, DEATH và ba giai đoạn tấn công.

```
if (!onGround) {
        if (Math.abs(velocityX) > 1f) {
            currentState = State.RUN;
        } else {
            currentState = velocityY > 0 ? State.JUMP : State.FALL;
        }
    } else {
        currentState = Math.abs(velocityX) > 1f ? State.RUN : State.IDLE;
    }

    updateRunSound(delta);
  }
```

Logic Chuyển đổi Trạng thái của Player: Hệ thống không chỉ dựa vào velocityY để xác định trạng thái nhảy hay rơi, mà còn xét đồng thời điều kiện tiếp đất (onGround) và vận tốc ngang (velocityX). Cụ thể, khi nhân vật đang ở trên không (!onGround), chương trình trước hết kiểm tra độ lớn của vận tốc ngang. Nếu Math.abs(velocityX) > 1f, nhân vật sẽ bị đưa vào trạng thái RUN dù đang ở trên không. Chỉ khi vận tốc ngang gần như bằng 0, hệ thống mới tiếp tục phân biệt giữa JUMP và FALL dựa trên dấu của

velocityY: nếu velocityY > 0 thì là JUMP, ngược lại là FALL. Khi nhân vật đứng trên mặt đất (onGround == true), trạng thái được xác định đơn giản hơn: nếu có chuyển động ngang thì là RUN, nếu không thì là IDLE.

```
idleTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Idle.png");
    runTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Run.png");
    jumpTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Jump.png");
    fallTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Fall.png");

    attackTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Attack.png");
    attack2Texture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Attack2.png");
    attackComboTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_AttackCombo.png");
    attackNoMoveTexture =
Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_AttackNoMovement.png");
    attack2NoMoveTexture =
Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Attack2NoMovement.png");
    attackComboNoMoveTexture =
Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_AttackComboNoMovement.png");
    hurtTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Hit.png");
    deathTexture = Assets.load("ASSETSGame/FreeKnight_v1/Colour2/NoOutline/120x80_PNGSheets/_Death.png");
```

Tích hợp Sprite Sheet: Các hoạt ảnh được cắt từ các tấm ảnh lớn (Sprite Sheets) với kích thước chuẩn 120x80 pixel cho mỗi khung hình.

Hệ thống Combo Tấn công Đa tầng: đây là điểm nhấn kỹ thuật trong lớp Player, hệ thống tấn công của người chơi không chỉ phụ thuộc vào việc nhấn phím tấn công trong khi đang giữ phím di chuyển, mà còn phụ thuộc vào trạng thái tiếp đất của nhân vật cho phép tạo ra các chuỗi đòn đánh liên hoàn đầy uy lực.

```
private void startAttack(int stage, InputState input) {
    isAttacking = true;
    attackSerial++;
    attackStage = Math.max(1, Math.min(stage, 3));
    queuedAttackStage = attackStage;
    attackWithMovement = onGround && (input.isKeyDown(KeyEvent.VK_A) || input.isKeyDown(KeyEvent.VK_D));
    stateTime = 0f;
    playAttackSound(attackStage);
  }
private State attackStageToState(int stage) {
    switch (stage) {
      case 2:
        return State.ATTACK2;
      case 3:
        return State.ATTACK3;
      default:
        return State.ATTACK1;
    }
  }
```

Cơ chế Xếp hàng Đòn đánh (Input Buffering): Game sử dụng hai biến điều hướng là attackStage (giai đoạn hiện tại) và queuedAttackStage (giai đoạn kế tiếp). Khi người chơi nhấn phím ENTER trong lúc một hoạt ảnh tấn công đang diễn ra, hệ

thống sẽ "ghi nhớ" lệnh này. Nếu hoạt ảnh hiện tại kết thúc và có lệnh chờ, người chơi sẽ ngay lập tức chuyển sang đòn đánh tiếp theo mà không bị khựng lại.

```
private Animation getCurrentAttackAnimation() {
    switch (attackStage) {
        case 2:
            return attackWithMovement ? attack2Animation : attack2NoMoveAnimation;
        case 3:
            return attackWithMovement ? attackComboAnimation : attackComboNoMoveAnimation;
        default:
            return attackWithMovement ? attackAnimation : attackNoMoveAnimation;
    }
}
```

Sự khác biệt về Di động (Movement-based Attacks): Nếu người chơi đứng yên khi nhấn tấn công, hệ thống sử dụng các bộ hoạt ảnh NoMove (như attackNoMoveAnimation).

Tấn công động: Nếu người chơi vừa di chuyển (A hoặc D) vừa tấn công, các hoạt ảnh chém lướt sẽ được kích hoạt, tạo ra sự linh hoạt trong giao tranh.

Attacking range của nhân vật Ngoài hệ thống hoạt ảnh và trạng thái chiến đấu, lớp Player còn cài đặt thêm các thành phần hỗ trợ quan trọng cho cơ chế gây sát thương và xử lý vòng đời nhân vật sau khi chết.

```
private static final float ATTACK_BOX_WIDTH = 24f;
private static final float ATTACK_BOX_HEIGHT = 22f;
```

Hai hằng số trên xác định kích thước của vùng gây sát thương (attack box) của người chơi. vùng này chỉ được sử dụng khi nhân vật đang thực hiện đòn đánh. Việc tách riêng attack box giúp hệ thống phân biệt rõ giữa va chạm thân thể và va chạm gây sát thương, từ đó tăng độ chính xác trong cơ chế chiến đấu.

```
public boolean overlapsAttackBox(float targetX, float targetY, float targetWidth, float targetHeight) {
    if (!isAttacking || dead) return false;
    if (stateTime < ATTACK_HIT_START || stateTime > ATTACK_HIT_END) return false;

    float attackWidth = ATTACK_BOX_WIDTH;
    float attackHeight = ATTACK_BOX_HEIGHT;
    float attackX = facingRight
        ? (getCollisionX() + COLLISION_WIDTH - 1f)
        : (getCollisionX() - attackWidth + 1f);
    float attackY = getCollisionY() + 4f;

    return attackX < targetX + targetWidth
        && attackX + attackWidth > targetX
        && attackY < targetY + targetHeight
        && attackY + attackHeight > targetY;
}
```

Đoạn mã trên cho thấy vùng tấn công có kích thước cố định 24x22 pixel và được đặt lệch về phía trước theo hướng nhìn của nhân vật. Điều này cho phép đòn đánh chỉ

tác động trong một vùng giới hạn phía trước người chơi, thay vì trên toàn bộ sprite, qua đó làm cho cơ chế cận chiến trở nên rõ ràng và hợp lý hơn.

Cơ chế Vật lý và Va chạm (Physics & Collision Logic):

Để đạt được độ chính xác cao trong trải nghiệm đi cảnh (platforming), cơ chế va chạm được tách biệt hoàn toàn với hình ảnh hiển thị.

```
public static final float SPRITE_WIDTH = 120f * 0.75f;
public static final float SPRITE_HEIGHT = 80f * 0.75f;
public static final float COLLISION_WIDTH = 18f;
public static final float COLLISION_HEIGHT = 30f;
public static final float COLLISION_OFFSET_X = (SPRITE_WIDTH - COLLISION_WIDTH) / 2f;
public static final float COLLISION_OFFSET_Y = 4f;
public static final float HEAD_SENSOR_HEIGHT = 4f;

public float getCollisionX() {
    return x + COLLISION_OFFSET_X;
}

public float getCollisionY() {
    return y + COLLISION_OFFSET_Y;
}
```

Hộp va chạm thân xác (Collision Box): để tăng độ chính xác trong xử lý va chạm, lớp Player sử dụng một hộp va chạm riêng có kích thước nhỏ hơn đáng kể so với kích thước hình ảnh hiển thị của nhân vật. Cụ thể, hộp va chạm có kích thước 18x30 pixel, trong khi sprite hiển thị có kích thước lớn hơn. Ngoài ra, hộp va chạm được căn giữa theo trục ngang thông qua công thức tính COLLISION_OFFSET_X, còn vị trí theo trục dọc được dịch lên một khoảng cố định COLLISION_OFFSET_Y = 4f.

```
private static final float ATTACK_HIT_START = 0.08f;
private static final float ATTACK_HIT_END = 0.20f;
```

Độ trễ thời gian (Frame-perfect timing): Sát thương không gây ra xuyên suốt toàn bộ hoạt ảnh mà chỉ xuất hiện trong khung thời gian cực ngắn (từ 0.08 giây đến 0.20 giây của hoạt ảnh).

```
public static final float GRAVITY = -900f;
```

Trọng lực và Gia tốc:

Người chơi chịu tác động của hằng số trọng lực mạnh (GRAVITY = -900f), tạo cảm giác rơi nhanh và chắc chắn, tránh hiện tượng "nhảy lơ lửng" thường thấy ở các game kém chất lượng.

```
private static final float HURT_DURATION = 0.32f;
if (hurtTimer > 0f) {
        hurtTimer -= delta;
```

```
        velocityX = 0f;
        if (velocityY < -600f) velocityY = -600f;
        velocityY += GRAVITY * delta;
        y += velocityY * delta;
        currentState = State.HURT;
        runStepTimer = 0f;
        if (hurtTimer <= 0f) {
            hurtTimer = 0f;
            stateTime = 0f;
        }
        return;
    }
public void takeDamage(float amount, String reason) {
    if (dead) return;
    hp = Math.max(0f, hp - amount);
    if (hp <= 0f) {
        die(reason);
        return;
    }
    endAttack();
    currentState = State.HURT;
    hurtTimer = HURT_DURATION;
    stateTime = 0f;
    playRandomDamageGrunt();
}
```

Kích hoạt trạng thái đau: Khi takeDamage được gọi, hurtTimer được gán bằng 0.32 giây, đồng thời ngắt mọi đòn tấn công đang dở và phát âm thanh đau. Khóa điều khiển: Trong suốt 0.32 giây này, velocityX = 0 khiến bạn không thể di chuyển (bị choáng/stun).

Hình ảnh 3.1: nhân vật rơi vào trạng thái State.HURT để hiển thị hoạt ảnh bị thương.

Hệ thống Sinh mệnh và Phản hồi (Health & Feedback):

Người chơi có tổng cộng 7 đơn vị máu (MAX_HP = 7f). Tương đương với 7 hình ảnh biểu thị lượng máu của người chơi.

Hình ảnh 3.2: Thanh máu người chơi.

```
public void takeDamage(float amount, String reason) {
    if (dead) return;
    hp = Math.max(0f, hp - amount);
    if (hp <= 0f) {
        die(reason);
        return;
    }
```

Cơ chế nhận sát thương: Khi trúng đòn từ quái vật (như OrcEnemy), phương thức takeDamage sẽ được gọi, trừ HP và kiểm tra điều kiện tử vong.

```
public void heal(float amount) {
    if (dead) return;
    hp = Math.min(MAX_HP, hp + amount);
}
```

Cơ chế hồi phục: Có khả năng tương tác với các thực thể khác như HealthPotion thông qua phương thức heal, nhưng không bao giờ vượt quá lượng máu tối đa.

Trạng thái Tử vong và Xử lý Hồi sinh: Khi lượng máu của người chơi giảm về 0, hệ thống không thực hiện toàn bộ quy trình tử vong và hồi sinh ngay trong một phương thức duy nhất, mà tách thành hai giai đoạn riêng biệt: xử lý tử vong trong lớp Player và xử lý chuyển màn hình/hồi sinh ở lớp quản lý màn chơi.

```
public void die(String reason) {
    if (dead) return;
    dead = true;
    deathReason = reason == null ? "enemy_hit" : reason;
    endAttack();
    currentState = State.DEATH;
    stateTime = 0f;
    velocityX = 0f;
    velocityY = 0f; }
public void resetForRespawn(float spawnX, float spawnY) {
    x = spawnX;
    y = spawnY;
    velocityX = 0f;
    velocityY = 0f;
    onGround = false;
    facingRight = true;
    hp = MAX_HP;
    dead = false;
    deathReason = "";
    airborneSinceGroundContact = false;
    runStepTimer = 0f;
```

```
    hurtTimer = 0f;
    currentState = State.IDLE;
    endAttack();
    stateTime = 0f; }
```

Trong lớp Player, phương thức die(String reason) chỉ đảm nhiệm việc đánh dấu nhân vật đã chết (dead = true), lưu nguyên nhân tử vong (deathReason), hủy đòn đánh đang diễn ra, chuyển trạng thái sang DEATH, đồng thời đặt toàn bộ vận tốc về 0. Điều này cho thấy lớp Player chỉ xử lý phần logic nội tại của thực thể người chơi khi tử vong, chứ chưa thực hiện chuyển màn hình hay hồi sinh.

```
if (!player.isDead()) {
    checkMapTeleport();
} else if (!deathScreenQueued && player.isDeathAnimationFinished()) {
    deathScreenQueued = true;
game.setScreen(new DeathScreen(game, 1, player.getDeathReason(),
getDeathMessage(player.getDeathReason())));
    return;
}
```

Ở lớp GameScreen, sau khi người chơi chết, hệ thống không lập tức hồi sinh mà chờ cho đến khi hoạt ảnh tử vong phát xong, được kiểm tra thông qua player.isDeathAnimationFinished(). Khi điều kiện này thỏa mãn, trò chơi mới chuyển sang DeathScreen. Cách tổ chức này cho thấy việc hiển thị hoạt ảnh tử vong và việc điều phối luồng màn hình được tách biệt rõ ràng.

```
public void startGameForPlayer(String playerName) {
    currentPlayerName = sanitizePlayerName(playerName);
    currentRunSeconds = 0f;
    runTimerActive = true;
    scoreSubmitted = false;
    startLevel(1);
}
public void startLevel(int level) {
    if (level == 3) {
        player.resetForRespawn(6f, 250f);
        setScreen(new TheEndScreen(this));
    } else if (level == 2) {
        player.resetForRespawn(40f, 140f);
        setScreen(new GameScreen2(this));
    } else {
```

```
    player.resetForRespawn(50f, 120f);
    setScreen(new GameScreen(this));
  }
}
```

Trong lớp Core, việc hồi sinh thực sự được thực hiện thông qua phương thức resetForRespawn(...), kết hợp với setScreen(...) để đưa người chơi trở lại màn chơi tương ứng. Như vậy, có thể khẳng định rằng logic hồi sinh hoặc chuyển màn hình không nằm trong Player.die(), mà được xử lý ở lớp điều phối cấp cao hơn.

Vậy, mô tả "phát hoạt ảnh kết thúc trước khi thực hiện logic hồi sinh (respawn)" trong mã nguồn được tách thành hai bước rõ ràng:

- Player.die() xử lý trạng thái tử vong của nhân vật;
- GameScreen và Core đảm nhiệm việc chờ hoạt ảnh kết thúc, chuyển màn hình và hồi sinh nhân vật.

3.1.2. Phần code OrcEnemy.

Lớp OrcEnemy đại diện cho loại quái vật cận chiến cơ bản trong game. Thực thể này được xây dựng với một hệ thống có khả năng tự động tuần tra, phát hiện mục tiêu và phản ứng linh hoạt với các đòn tấn công từ người chơi.

Hệ thống Trạng thái và Hành vi:

```
if (x < patrolMinX) {
        moveDir = 1f;
    } else if (x > patrolMaxX) {
        moveDir = -1f;
    }

    velocityX = moveDir * SPEED;
    facingRight = moveDir >= 0f;
    if (moveDir < 0f && x <= patrolMinX) {
        x = patrolMinX;
        moveDir = 1f;
        patrolPauseTimer = PATROL_IDLE_TIME;
        stateTime = 0f;
        state = State.IDLE;
        velocityX = 0f;
    } else if (moveDir > 0f && x >= patrolMaxX) {
        x = patrolMaxX;
        moveDir = -1f;
        patrolPauseTimer = PATROL_IDLE_TIME;
        stateTime = 0f;
        state = State.IDLE;
        velocityX = 0f;
    }
  }
```

Trạng thái Tuần tra (Patrol): Khi không có người chơi trong tầm mắt, Orc di chuyển qua lại giữa patrolMinX và patrolMaxX với tốc độ ổn định (SPEED = 36f). Khi chạm đến giới hạn biên, nó sẽ dừng lại trong một khoảng thời gian nghỉ (PATROL_IDLE_TIME = 0.8s) để tạo cảm giác tự nhiên trước khi đổi hướng.

```
    private static final float CHASE_RANGE_X = 180f;
private static final float SPEED = 36f;
  private static final float CHASE_SPEED = 52f;
private boolean isPlayerInAttackRange(Player player) {
  float rangeX = ATTACK_RANGE_X;
  float rangeY = ATTACK_RANGE_Y;
  return Math.abs((player.getCollisionX() + Player.COLLISION_WIDTH * 0.5f) - (getCollisionX() +
COLLISION_WIDTH * 0.5f)) <= rangeX
      && Math.abs((player.getCollisionY() + Player.COLLISION_HEIGHT * 0.5f) - (getCollisionY() +
COLLISION_HEIGHT * 0.5f)) <= rangeY;
}

private boolean isPlayerInChaseRange(Player player) {
  float rangeX = CHASE_RANGE_X;
  float rangeY = CHASE_RANGE_Y;
  return Math.abs((player.getCollisionX() + Player.COLLISION_WIDTH * 0.5f) - (getCollisionX() +
COLLISION_WIDTH * 0.5f)) <= rangeX
      && Math.abs((player.getCollisionY() + Player.COLLISION_HEIGHT * 0.5f) - (getCollisionY() +
COLLISION_HEIGHT * 0.5f)) <= rangeY;
}
```

Trạng thái Truy đuổi (Chase): Khi người chơi đi vào vùng nhận diện (CHASE_RANGE_X = 180f), Orc sẽ tăng tốc độ từ SPEED = 36f lên CHASE_SPEED = 52f để áp sát mục tiêu. Nhờ đó, Orc chỉ phản ứng khi người chơi thực sự nằm trong vùng tương tác phù hợp, tránh tình trạng phát hiện hoặc tấn công sai lệch khi chênh lệch độ cao quá lớn.

```
private static final float ATTACK_RANGE_X = 28f;
```

Trạng thái Tấn công (Attack): Khi khoảng cách đủ gần (ATTACK_RANGE_X = 28f), Orc sẽ dừng di chuyển và thực hiện chuỗi đòn đánh.

Cơ chế Chiến đấu và Xử lý Va chạm :

Hệ thống chiến đấu của Orc được thiết kế để tạo ra thử thách vừa phải cho người chơi.

```
private final Animation attack1Animation;
private final Animation attack2Animation;
if (playerInAttackRange && attackCooldown <= 0f) {
    attacking = true;
    attackHitApplied = false;
    useSecondAttack = !useSecondAttack;
    stateTime = 0f;
    state = State.ATTACK;
    OneShotSound.play(ORC_ATTACK_SOUND);
    return;
}
if (attacking) {
    state = State.ATTACK;
    Animation attackAnim = useSecondAttack ? attack2Animation : attack1Animation;
    velocityX = 0f;
```

Đòn đánh xen kẽ: Orc sở hữu hai kiểu hoạt ảnh tấn công khác nhau (attack1 và attack2). Việc lựa chọn giữa hai đòn đánh này được điều khiển bởi biến useSecondAttack. Cách xử lý này tạo ra cơ chế xen kẽ giữa hai hoạt ảnh tấn công, giúp hành vi chiến đấu của Orc đa dạng hơn mà vẫn giữ được tính đơn giản trong cài đặt.

```
private static final float ATTACK_HIT_TIME = 0.2f;
```

Hộp tấn công (Attack Box): Tương tự người chơi, sát thương của Orc chỉ được kích hoạt tại một thời điểm chính xác trong hoạt ảnh (ATTACK_HIT_TIME = 0.2). Vị trí của Attack Box được tính toán dựa trên hướng nhìn của Orc để đảm bảo tính chính xác về mặt hình ảnh.

Phản hồi Sát thương và Tương tác (Hurt & Death Logic):

```
private void handleHitFromPlayer(Player player) {
    if (isDead() || hurtTimer > 0f || !player.isAttacking()) return;
    if (player.getAttackSerial() == lastPlayerAttackSerial) return;
    if (player.overlapsAttackBox(getCollisionX(), getCollisionY(), COLLISION_WIDTH, COLLISION_HEIGHT)) {
        lastPlayerAttackSerial = player.getAttackSerial();
        playHitSound(player.getAttackStage());
        hp -= 1f;
        attacking = false;
        attackHitApplied = false;
        stateTime = 0f;
        if (hp <= 0f) {
            state = State.DEATH;
        } else {
```

```
            hurtTimer = 0.24f;
            state = State.HURT;
        }
    }
}
```

Xử lý trúng đòn: Phương thức handleHitFromPlayer kiểm tra va chạm với AttackBox của người chơi. Để tránh việc một đòn đánh gây sát thương nhiều lần, Orc sử dụng lastPlayerAttackSerial để ghi nhớ mã định danh của đòn đánh vừa nhận.

Hiệu ứng Hurt: Khi bị trúng đòn, Orc bị đẩy vào trạng thái HURT trong 0.24s, trong thời gian này mọi hoạt động di chuyển và tấn công của nó đều bị đình chỉ.

Các thông số và cơ chế bổ sung của OrcEnemy:

Ngoài các trạng thái tuần tra, truy đuổi và tấn công đã được mô tả, lớp OrcEnemy còn chứa nhiều thông số và cơ chế quan trọng khác ảnh hưởng trực tiếp đến hành vi chiến đấu, khả năng di chuyển và vòng đời của quái vật trong trò chơi.

```
private static final float SPEED = 36f;
private static final float CHASE_SPEED = 52f;
private static final float ATTACK_COOLDOWN = 0.7f;
private static final float ATTACK_RANGE_X = 28f;
private static final float ATTACK_RANGE_Y = 22f;
private static final float CHASE_RANGE_X = 180f;
private static final float CHASE_RANGE_Y = 60f;
private float hp = 3f;
private float attackCooldown = 0f;
```

Đoạn mã trên cho thấy Orc không chỉ có các ngưỡng phát hiện và tấn công theo trục ngang, mà còn được giới hạn thêm theo trục dọc. Cụ thể, để thực hiện tấn công, người chơi phải nằm trong phạm vi ATTACK_RANGE_X = 28f và ATTACK_RANGE_Y = 22f; để bị phát hiện và truy đuổi, người chơi phải nằm trong phạm vi CHASE_RANGE_X = 180f và CHASE_RANGE_Y = 60f. Điều này cho thấy cơ chế nhận diện mục tiêu của Orc được xây dựng theo vùng hai chiều, thay vì chỉ xét khoảng cách ngang. Ngoài ra, Orc có lượng máu khởi tạo là 3f, đồng nghĩa với việc cần nhiều hơn một đòn đánh để tiêu diệt. Biến ATTACK_COOLDOWN = 0.7f được dùng để tạo khoảng nghỉ giữa các lần ra đòn, giúp nhịp tấn công của kẻ địch trở nên hợp lý hơn.

Khả năng nhảy của Orc:

Bên cạnh khả năng đi bộ và tấn công cận chiến, lớp OrcEnemy còn hỗ trợ hành vi nhảy thông qua phương thức tryJump().

```
private static final float JUMP_FORCE = 235f;
private static final float JUMP_COOLDOWN = 0.35f;
private float jumpCooldown = 0f;
public boolean onGround = false;
public boolean tryJump() {
    if (!onGround || jumpCooldown > 0f || isDead() || hurtTimer > 0f || attacking) {
        return false;
    }
    velocityY = JUMP_FORCE;
    onGround = false;
    jumpCooldown = JUMP_COOLDOWN;
    return true;
}
```

Phương thức này cho phép Orc thực hiện một cú nhảy khi đang đứng trên mặt đất, không trong thời gian hồi nhảy (jumpCooldown), không bị thương, không đang tấn công và chưa chết. Khi điều kiện hợp lệ, vận tốc theo trục dọc được gán bằng JUMP_FORCE = 235f và bộ đếm hồi nhảy được đặt lại. Điều này cho thấy Orc không chỉ là một thực thể di chuyển ngang đơn giản, mà còn có thể tham gia vào môi trường platformer với khả năng vượt địa hình.

Cơ chế loại bỏ kẻ địch và đánh dấu rơi vật phẩm:

Sau khi bị tiêu diệt, Orc không lập tức biến mất ngay mà được giữ lại cho đến khi hoạt ảnh chết hoàn tất. Đồng thời, hệ thống cũng quản lý riêng việc quái đã rơi vật phẩm hay chưa.

```
private boolean potionDropped = false;
public boolean isPotionDropped() {
    return potionDropped;
}

public void markPotionDropped() {
    potionDropped = true;
}

public boolean isRemovable() {
    return isDead() && deathAnimation.isFinished(stateTime);
}
```

Biến potionDropped dùng để đánh dấu việc Orc đã thả vật phẩm hồi máu hay chưa, nhằm tránh trường hợp cùng một kẻ địch rơi vật phẩm nhiều lần. Phương thức markPotionDropped() được gọi sau khi vật phẩm đã được sinh ra, còn isPotionDropped() giúp màn chơi kiểm tra trạng thái này. Bên cạnh đó, isRemovable() chỉ trả về true khi Orc đã chết và hoạt ảnh tử vong đã phát xong. Nhờ vậy, quá trình xóa quái khỏi danh sách quản lý diễn ra đúng thời điểm, không làm mất hiệu ứng hình ảnh.

```
if (orc.isRemovable() && !orc.isPotionDropped()) {
    float dropX = orc.getCollisionX() + orc.getCollisionWidth() / 2f;
    float dropY = orc.getCollisionY();
    potions.add(new HealthPotion(dropX, dropY));
    OneShotSound.play(POTION_SPAWN_SOUND);
    orc.markPotionDropped();
}
if (orc.isRemovable()) {
    orcs.remove(i);
}
```

Trong màn chơi, sau khi isRemovable() trả về đúng, hệ thống sẽ sinh HealthPotion nếu Orc chưa từng thả vật phẩm, sau đó đánh dấu bằng markPotionDropped(). Cuối cùng, Orc mới bị xóa khỏi danh sách thực thể. Cách tổ chức này bảo đảm tính nhất quán giữa hiệu ứng chết, phần thưởng rơi ra và vòng đời của đối tượng.

3.1.3 Thực thể Vật phẩm (Health Potion).

Lớp HealthPotion quản lý các vật phẩm hồi máu rơi ra sau khi tiêu diệt kẻ địch, tập trung vào mô phỏng vật lý đơn giản và trải nghiệm nhặt vật phẩm mượt mà.

Hình ảnh 3.3: vật phẩm hồi máu

```
this.velocityX = ThreadLocalRandom.current().nextFloat() * 60f - 30f;
this.velocityY = 160f + ThreadLocalRandom.current().nextFloat() * 60f;
```

Mô phỏng Vật lý (Projectile Motion): Khi được tạo ra, bình máu không đứng yên mà có vận tốc ban đầu ngẫu nhiên (velocityX từ -30 đến 30, velocityY từ 160 đến 220).

```
private static final float PICKUP_DELAY = 0.35f;
```

Cơ chế Nhặt vật phẩm (Pickup Logic): * Độ trễ (Pickup Delay): Hệ thống áp dụng một khoảng thời gian chờ 0.35s trước khi cho phép người chơi nhặt. Điều này ngăn chặn việc vật phẩm bị nhặt ngay lập tức khi người chơi đang tấn công áp sát.

```
if (y <= groundY) {
        y = groundY;
        velocityY = 0f;
        velocityX = 0f;
        grounded = true;
    }
```

Trạng thái nghỉ: Khi chạm mặt đất (y <= groundY), các vận tốc được triệt tiêu và biến grounded chuyển sang true để dừng tính toán vật lý, giúp tối ưu hiệu năng.

Cơ chế nhặt vật phẩm và hồi máu:

Trong vòng lặp cập nhật của màn chơi, hệ thống duyệt qua danh sách các bình máu đang tồn tại, sau đó kiểm tra các điều kiện: vùng va chạm của vật phẩm giao với hộp va chạm của người chơi, và người chơi chưa ở trạng thái chết. Khi các điều kiện này được thỏa mãn, trò chơi thực hiện hồi máu cho người chơi thông qua phương thức player.heal(healAmount).

```
if (!player.isDead()) {
  for (int i = potions.size() - 1; i >= 0; i--) {
    HealthPotion potion = potions.get(i);
    if (potion.canBePickedUp()
      && potion.getX() < player.getCollisionX() + Player.COLLISION_WIDTH
      && potion.getX() + potion.getWidth() > player.getCollisionX()
      && potion.getY() < player.getCollisionY() + Player.COLLISION_HEIGHT
      && potion.getY() + potion.getHeight() > player.getCollisionY()) {
      int healAmount = ThreadLocalRandom.current().nextInt(1, 3);
      player.heal(healAmount);
      OneShotSound.play(POTION_PICKUP_SOUND);
      potions.remove(i);
    }
  }
```

Đáng chú ý là lượng máu hồi không phải giá trị cố định, mà được sinh ngẫu nhiên trong khoảng từ 1 đến 2 HP thông qua lệnh

ThreadLocalRandom.current().nextInt(1, 3). Cách thiết kế này giúp tăng tính biến thiên trong trải nghiệm chơi, đồng thời tạo cảm giác phần thưởng linh hoạt hơn thay vì hoàn toàn cứng nhắc.

3.2 Xây dựng Hệ thống Màn hình và Giao diện người dùng

Hệ thống màn hình đóng vai trò quản lý vòng đời của trò chơi, từ menu bắt đầu đến các màn chơi chính và các thông báo kết thúc. Việc sử dụng Screen giúp trò chơi dễ dàng chuyển đổi trạng thái mà không làm xáo trộn logic hệ thống.

3.2.1 Quản lý Màn hình Chính (Start Screen & Game Screen).

Màn hình chính là điểm giao tiếp đầu tiên với người chơi, được thiết kế để điều hướng các chức năng hệ thống và khởi tạo dữ liệu phiên chơi.

Màn hình Bắt đầu (StartScreen): Lớp StartScreen không chỉ quản lý các lựa chọn menu đơn lẻ, mà còn tổ chức giao diện theo nhiều lớp trạng thái khác nhau. Trong mã nguồn, lớp này sử dụng đồng thời ba enum khác nhau để quản lý ba nhóm trạng thái riêng biệt: trạng thái hiển thị tổng quát, lựa chọn menu chính và lựa chọn độ phân giải:

```
public class StartScreen implements Screen {
    private enum ViewMode {
        HOME,
        SUBPAGE,
        PLAYER_NAME
    }

    private enum MenuOption {
        PLAY,
        RESOLUTION,
        INTRO,
        GUIDE,
        LEADERBOARD,
        CREDITS,
        EXIT
    }

    private enum ResolutionOption {
        SMALL,
        MEDIUM,
        LARGE,
        FULL_SCREEN
```

Trong đó, ViewMode là thành phần thể hiện đúng vai trò của một "máy trạng thái con" bên trong StartScreen. Biến này quyết định màn hình hiện đang ở chế độ nào: giao diện chính (HOME), giao diện nội dung phụ (SUBPAGE) hay giao diện nhập tên

người chơi (PLAYER_NAME). Đây là lớp trạng thái bao quát nhất, chi phối cách xử lý đầu vào và cách hiển thị toàn bộ giao diện.

```
private ViewMode viewMode = ViewMode.HOME;
private int keyboardFocusIndex = MenuOption.INTRO.ordinal();
private MenuOption currentPanel = MenuOption.INTRO;
private int resolutionFocusIndex;
```

Ngược lại, MenuOption chỉ dùng để xác định mục nào đang được chọn trong menu chính, chẳng hạn như PLAY, GUIDE, LEADERBOARD hay EXIT. Enum này không đại diện cho trạng thái hiển thị tổng thể của màn hình, mà chỉ là tập các chức năng mà người dùng có thể truy cập.

```
private void activate(MenuOption option) {
    switch (option) {
        case PLAY:
            playerNameInput.setLength(0);
            if (!game.getCurrentPlayerName().isBlank()) {
                playerNameInput.append(game.getCurrentPlayerName());
            }
            playerNameError = "";
            viewMode = ViewMode.PLAYER_NAME;
            break;
        case EXIT:
            System.exit(0);
            break;
        default:
            viewMode = ViewMode.SUBPAGE;
            currentPanel = option;
            keyboardFocusIndex = option.ordinal();
            if (option == MenuOption.RESOLUTION) {
                resolutionFocusIndex = 0;
            }
            break;
    }
}
```

Đoạn mã trên cho thấy MenuOption chỉ đóng vai trò đầu vào để làm thay đổi ViewMode. Khi người chơi chọn PLAY, hệ thống chuyển viewMode sang PLAYER_NAME; khi chọn các mục nội dung khác, hệ thống chuyển sang SUBPAGE. Như vậy, ViewMode mới là thành phần trực tiếp điều phối luồng hoạt động của màn hình.

```
private void updateInput() {
    InputState input = game.getInput();

    if (viewMode == ViewMode.PLAYER_NAME) {
        updatePlayerNameInput(input);
        return;
    }

    if (viewMode == ViewMode.SUBPAGE) {
        if (input.isKeyJustPressed(KeyEvent.VK_ESCAPE) ||
input.isKeyJustPressed(KeyEvent.VK_BACK_SPACE)) {
            returnToHome();
```

```
        return;
    }
    if (currentPanel == MenuOption.RESOLUTION) {
        updateResolutionInput(input);
        return;
    }
    if (input.isKeyJustPressed(KeyEvent.VK_ENTER)) {
        returnToHome();
    }
    return;
}
```

Trong xử lý đầu vào, mã nguồn trước hết kiểm tra viewMode để quyết định hướng xử lý tương ứng. Điều này khẳng định rằng logic chính của StartScreen được tổ chức theo trạng thái hiển thị (ViewMode), chứ không chỉ theo danh sách lựa chọn menu.

Quản lý cấu hình: Cho phép thay đổi độ phân giải màn hình trực tiếp thông qua ResolutionOption (Small, Medium, Large, Full Screen), giúp tối ưu hóa hiệu suất trên các thiết bị khác nhau.

Cơ chế nhập tên người chơi trước khi bắt đầu:

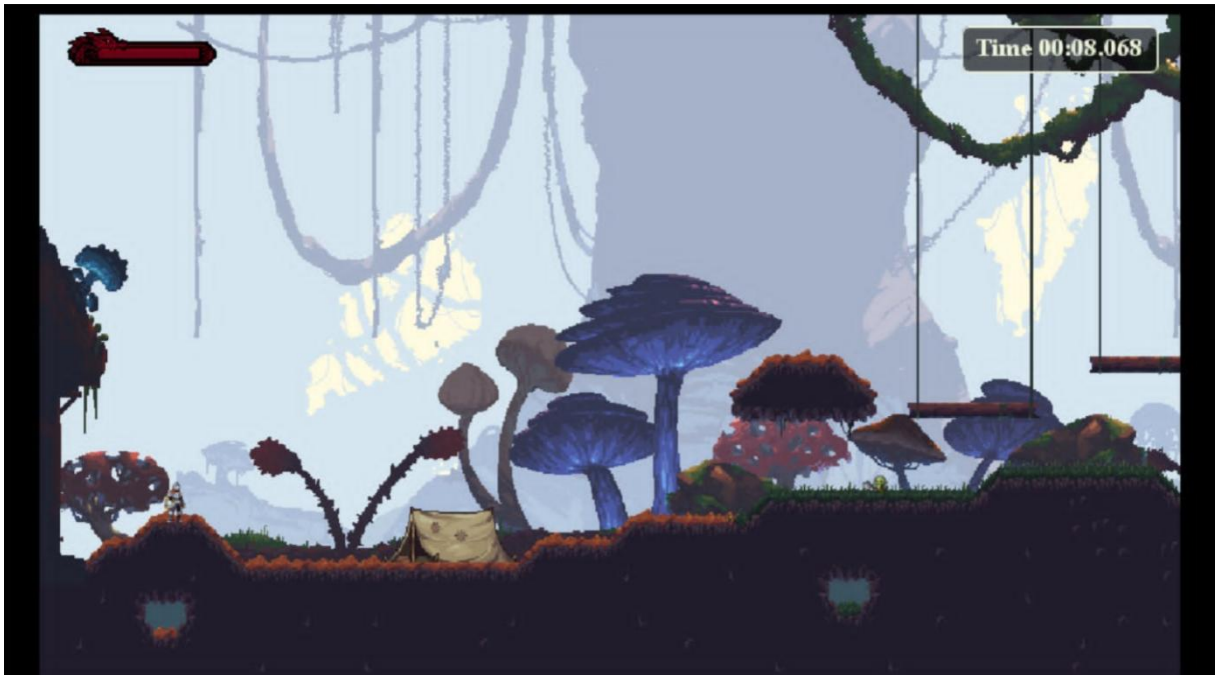
Khi bắt đầu màn chơi, Game yêu cầu người dùng nhập tên trước khi bắt đầu phiên chơi mới. Cơ chế này được tích hợp trực tiếp trong StartScreen.

```
private enum ViewMode {
    HOME,
    SUBPAGE,
    PLAYER_NAME
}
private final StringBuilder playerNameInput = new StringBuilder();
private String playerNameError = "";
```

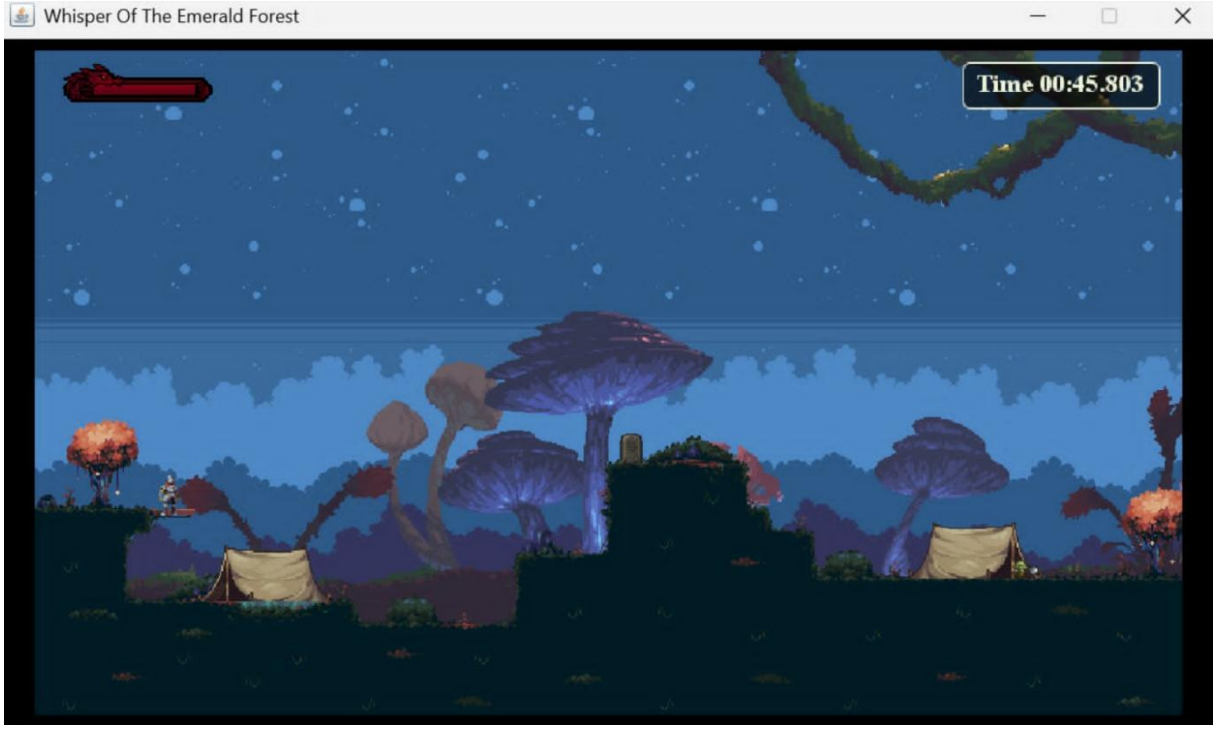
Dữ liệu tên người chơi được lưu trong playerNameInput, trong khi các lỗi nhập liệu được phản hồi qua playerNameError.

Hình ảnh 3.4: Nhập tên người chơi

Màn hình Chơi game (GameScreen & GameScreen2): Màn hình GameScreen là thành phần cốt lõi của trò chơi, chịu trách nhiệm quản lý vòng lặp logic (Update) và hiển thị (Render) toàn bộ các thực thể, bản đồ và giao diện người dùng trong quá trình chơi.



Hình ảnh 3.5: GameScreen



Hình ảnh 3.6 : Gamescreen2

```
mapRenderer = new TiledMapRenderer(map);
player = game.player;
groundLayer = map.getLayer("ground");
waterLayer = findWaterLayer();
```

Quản lý logic quan trọng như: Kiểm tra va chạm giữa người chơi và gạch (tiles), cập nhật vị trí Camera theo nhân vật, và xử lý âm thanh môi trường (BGS Loops).

Cơ chế fixed timestep trong vòng lặp cập nhật:

Một chi tiết kỹ thuật quan trọng trong GameScreen và GameScreen2 là việc hai lớp này không cập nhật gameplay hoàn toàn theo thời gian thực của từng khung hình, mà sử dụng cơ chế fixed timestep để chuẩn hóa nhịp mô phỏng. Cụ thể, hệ thống định nghĩa một bước thời gian cố định STEP = 1f / 60f, tương ứng với tần số cập nhật logic khoảng 60 lần mỗi giây. Đồng thời, biến accumulator được dùng để tích lũy phần thời gian thực đã trôi qua giữa các lần render.

```
private static final float STEP = 1f / 60f;
private float accumulator = 0f;
public void update(float delta) {
    if (!pauseOverlay.isPaused()) {
        game.addPlayTime(delta);
        accumulator += Math.min(delta, 0.25f);
        while (accumulator >= STEP) {
            player.update(STEP, input);
            player.x += player.velocityX * STEP;
            handlePlayerCollision(player);
```

```
// Cap nhat quai vat, vat pham vac ac nghiep vu gameplay khac

        accumulator -= STEP;
    }
  }
}
```

Cơ chế này cho phép toàn bộ các thành phần gameplay như người chơi, kẻ địch, vật phẩm, va chạm và hiệu ứng vật lý luôn được cập nhật theo cùng một đơn vị thời gian cố định, thay vì phụ thuộc trực tiếp vào tốc độ render của máy. Nhờ đó, hành vi của trò chơi trở nên ổn định hơn trên các cấu hình khác nhau, hạn chế sai lệch vật lý hoặc tốc độ di chuyển bất thường khi số khung hình thay đổi. Ngoài ra, việc giới hạn delta tối đa bằng Math.min(delta, 0.25f) còn giúp hệ thống tránh hiện tượng tích lũy thời gian quá lớn khi trò chơi bị khựng hoặc tạm dừng ngắn hạn

3.2.2. Hệ thống Lớp phủ Giao diện (UI Overlays).

Trong thiết kế trò chơi, các lớp phủ (Overlays) đóng vai trò cung cấp thông tin hoặc tùy chọn điều khiển mà không làm gián đoạn hoàn toàn luồng dữ liệu của màn hình hiện tại. Lớp PauseOverlay là một thành phần giao diện động, cho phép người chơi tương tác với hệ thống khi trò chơi đang tạm dừng.

```
if (!paused) {
  if (input.isKeyJustPressed(KeyEvent.VK_ESCAPE)) {
    paused = true;
    selectedIndex = OPTION_CONTINUE;
  }
  return false;
}
drawOption(g, camera, continueBounds, "Continue", selectedIndex == OPTION_CONTINUE);
drawOption(g, camera, restartBounds, "Restart", selectedIndex == OPTION_RESTART);
if (allowHome) {
drawOption(g, camera, homeBounds, "Home", selectedIndex == OPTION_HOME);
}
```

Tạm dừng (PauseOverlay): Xuất hiện khi người chơi nhấn ESC. Nó cung cấp các tùy chọn Continue, Restart, hoặc quay lại trang chủ (Home). Logic điều khiển sử dụng phím mũi tên và ENTER để lựa chọn.

```
public GameScreen(Core game) {
  this.game = game;
  this.pauseOverlay = new PauseOverlay(game, 1);
  this.transitionBlockedOverlay = new MessageOverlay(
    "You Have To Slay All the Enemy",
    "Enter to continue"
  );
}
```

Đoạn mã trên cho thấy GameScreen khởi tạo sẵn một lớp phủ tạm dừng (PauseOverlay) và một lớp phủ thông báo (MessageOverlay) dùng để ngăn người chơi

chuyển màn khi chưa hoàn thành điều kiện cần thiết. Điều này thể hiện rằng giao diện của trò chơi không chỉ gồm các màn hình toàn phần, mà còn bao gồm các lớp giao diện phụ xuất hiện chồng lên màn chơi.

```
if (transitionBlockedOverlay.isVisible()) {
    transitionBlockedOverlay.update(input);
    return;
}
if (pauseOverlay.update(camera, player, input)) {
    return;
}
```

Trong vòng lặp cập nhật, hệ thống ưu tiên xử lý trạng thái của các lớp phủ này trước khi chạy logic chính của màn chơi. Nếu thông báo đang hiển thị hoặc trò chơi đang ở trạng thái tạm dừng, quá trình cập nhật bình thường sẽ bị chặn lại. Cách tổ chức này giúp phân tách rõ ràng giữa logic gameplay và logic giao diện phụ trợ.

Cơ chế chuyển màn giữa các bản đồ:

Hệ thống màn hình của trò chơi còn bao gồm luồng chuyển tiếp giữa nhiều bản đồ và màn kết thúc, cụ thể theo trình tự GameScreen -> GameScreen2 -> TheEndScreen.

```
    if (player.x > worldWidth - Player.SPRITE_WIDTH) {
        if (!canTeleportToAnotherMap()) {
            player.x = worldWidth - Player.SPRITE_WIDTH;
            player.velocityX = 0f;
            transitionBlockedOverlay.show();
            return;
        }
        isTeleporting = true;
        player.x = MAP2_SPAWN_X;
        player.y = MAP2_SPAWN_Y;
        player.velocityX = 0f;
        player.velocityY = 0f;
        game.setScreen(new GameScreen2(game));
    }
}
```

Trong GameScreen, khi người chơi đi tới ranh giới chuyển cảnh, hệ thống sẽ đặt lại vị trí spawn phù hợp và chuyển sang GameScreen2.

```
public void startLevel(int level) {
    if (level == 3) {
        player.resetForRespawn(6f, 250f);
        setScreen(new TheEndScreen(this));
    } else if (level == 2) {
        player.resetForRespawn(40f, 140f);
        setScreen(new GameScreen2(this));
    } else {
        player.resetForRespawn(50f, 120f);
        setScreen(new GameScreen(this));
```

```
    }
}
```

Ở mức điều phối cao hơn, lớp Core quản lý việc mở từng màn chơi thông qua startLevel(int level). Điều này cho thấy hệ thống không chỉ có một màn chơi duy nhất, mà vận hành như một chuỗi màn hình liên kết với nhau, trong đó mỗi màn có vị trí khởi tạo riêng cho người chơi.

3.2.3. Xử lý Trạng thái Kết thúc (Death Screen & The End Screen).

Màn hình báo tử (DeathScreen) đóng vai trò quan trọng trong việc xử lý trạng thái thất bại của người chơi. Nó cung cấp phản hồi trực quan về nguyên nhân thất bại, đồng thời cho phép người chơi nhanh chóng quay lại trò chơi hoặc trở về menu chính.

```
public DeathScreen(Core game, int retryLevel, String deathReason, String deathMessage) {
    this.game = game;
    this.retryLevel = retryLevel;
    this.deathReason = deathReason;
    this.deathMessage = deathMessage;
    if (deathReason != null && deathReason.equalsIgnoreCase("drown")) {
        this.backgroundTexture = Assets.load("water death.png");
    } else {
        this.backgroundTexture = Assets.load("beat_death.png");
    }
    this.font = new Font("Serif", Font.BOLD, 20);
    this.retryButtonBounds = new Rectangle2D.Float();
    this.startButtonBounds = new Rectangle2D.Float();
}
```

Đoạn mã trên là hàm khởi tạo của lớp DeathScreen, đóng vai trò thiết lập các thông số cơ bản khi người chơi thất bại. Đầu tiên, nó tiếp nhận và lưu trữ các dữ liệu quan trọng như cấp độ để chơi lại (retryLevel) và thông điệp thông báo nguyên nhân cái chết. Điểm đáng chú ý là logic kiểm tra deathReason để tự động thay đổi hình nền tương ứng, giúp tăng tính biểu cảm trực quan cho trò chơi. Tiếp theo, mã nguồn khởi tạo font chữ hệ thống và các đối tượng hình học Rectangle2D để xác định vị trí tương tác cho các nút bấm. Cuối cùng, việc chuẩn bị kỹ lưỡng các tài nguyên này ngay từ đầu giúp màn hình báo tử hiển thị mượt mà và phản hồi chính xác hành động của người dùng.

```
    if (input.isKeyJustPressed(KeyEvent.VK_A)) {
        selectedOption = DeathOption.RETRY;
        return;
    }
    if (input.isKeyJustPressed(KeyEvent.VK_D)) {
        selectedOption = DeathOption.MENU;
        return;
    }
```

Đoạn mã này xử lý logic điều hướng lựa chọn trên màn hình bằng cách lắng nghe sự kiện từ bàn phím. Khi người chơi nhấn phím A hoặc D, biến selectedOption sẽ cập

nhật trạng thái tương ứng giữa việc thử lại hoặc quay về thực đơn chính, tạo ra cơ chế tương tác linh hoạt và trực quan để người dùng quyết định bước tiếp theo sau khi thất bại.

Hình ảnh 3.7 : Màn hình kết thúc.

Màn hình Kết thúc (TheEndScreen): Đánh dấu sự hoàn thành của hành trình. Tích hợp các hiệu ứng hình ảnh đặc biệt như Emerald Heart và xử lý âm thanh chiến thắng.Cung cấp các thông số tổng kết và cho phép người chơi quay lại màn hình chính một cách mượt mà.

Tích hợp bảng xếp hạng qua LeaderboardService:

Một thành phần quan trọng khác của giao diện là bảng xếp hạng, được kết nối với lớp dịch vụ LeaderboardService.

```
private static final String SUPABASE_PROJECT_URL = "https://tvmfauertbelxqngnwof.supabase.co";
private static final String SUPABASE_API_KEY = "sb_publishable_fW9iKPC_49x00nh5nEWiqw_dItziah-";
private final LeaderboardService leaderboardService;
public Core() {
    this.player = new Player(80f, 120f);
    this.leaderboardService = new LeaderboardService(SUPABASE_PROJECT_URL, SUPABASE_API_KEY);
}
```

Trong Core, đối tượng LeaderboardService được khởi tạo để đảm nhiệm việc giao tiếp dữ liệu với hệ thống xếp hạng. Điều này cho thấy chức năng leaderboard không phải là một phần hiển thị tĩnh, mà là một hệ thống dịch vụ có trao đổi dữ liệu.

```
public void refreshLeaderboard() {
    leaderboardService.fetchTopScoresAsync(10);
}
public void submitCurrentScore() {
    if (scoreSubmitted || currentPlayerName.isBlank()) {
        runTimerActive = false;
        return;
    }
    runTimerActive = false;
    scoreSubmitted = true;
    leaderboardService.submitScoreAsync(currentPlayerName, currentRunSeconds);
}
```

Các phương thức trên thể hiện hai chức năng chính của leaderboard: tải dữ liệu bảng xếp hạng và gửi điểm của người chơi sau khi hoàn thành phiên chơi. Việc sử dụng các phương thức bất đồng bộ (fetchTopScoresAsync, submitScoreAsync) giúp tránh làm gián đoạn giao diện trong khi hệ thống đang trao đổi dữ liệu.

```
public List<LeaderboardEntry> getLeaderboardEntries() {
    return leaderboardService.getCachedEntries();
}
public String getLeaderboardStatus() {
    return leaderboardService.getStatusMessage();
}
```

Bên trong LeaderboardService, dữ liệu sau khi tải về được lưu trong bộ nhớ đệm (cachedEntries) cùng với thông báo trạng thái (statusMessage). Điều này cho phép các màn hình như StartScreen hoặc TheEndScreen đọc và hiển thị dữ liệu bảng xếp hạng mà không phải trực tiếp thao tác với kết nối mạng.

3.3 Hệ thống Quản lý Bản đồ và Va chạm môi trường

Hệ thống bản đồ được xây dựng dựa trên sự phối hợp giữa cấu trúc dữ liệu TiledMap và bộ lọc hiển thị TiledMapRenderer, cho phép chuyển tải các tệp tin từ phần mềm Tiled thành môi trường tương tác thực tế.

3.3.1 Cấu trúc dữ liệu TiledMap và Tileset.

```
public int getWidth() {
    return width;
}

public int getHeight() {
    return height;
}
public int getTileWidth() {
    return tileWidth;
}
public int getTileHeight() {
    return tileHeight;
}
```

Kích thước bản đồ (width, height): Xác định số lượng ô gạch theo chiều ngang và chiều dọc. Kích thước ô gạch (tileWidth, tileHeight): Xác định độ phân giải của mỗi đơn vị ô vuông

```
public TiledLayer getLayer(String name) {
    for (TiledLayer layer : layers) {
        if (layer.getName() != null && layer.getName().equalsIgnoreCase(name)) {
            return layer;
        }
    }
    return null;
}
```

Kiểm tra điều kiện: Với mỗi lớp, nó kiểm tra xem tên lớp có tồn tại và có trùng khớp với tham số name hay không (không phân biệt chữ hoa chữ thường nhờ equalsIgnoreCase).Trả về kết quả: Nếu tìm thấy, nó sẽ trả về đối tượng lớp đó để lập trình viên có thể thao tác (như xử lý va chạm hoặc vẽ hình); nếu không tìm thấy sau khi duyệt hết danh sách, nó trả về null.

```
public Tileset getTilesetForGid(int gid) {
    Tileset match = null;
    for (Tileset tileset : tilesets) {
        if (gid >= tileset.firstGid && gid <= tileset.getLastGid()) {
            if (match == null || tileset.firstGid > match.firstGid) {
                match = tileset;
            }
        }
    }
    return match;
}
```

Cơ chế GID (Global Identifier): Bản đồ sử dụng hệ thống ID toàn cầu để xác định các ô gạch (tiles). Phương thức getTilesetForGid(int gid) cho phép hệ thống truy vấn chính xác ô gạch đó thuộc về bộ tài nguyên (Tileset) nào dựa trên dải ID mà Tileset đó nắm giữ.

```
int srcX = (localId % tileset.columns) * (tileset.tileWidth + tileset.spacing) + tileset.margin;
int srcY = (localId / tileset.columns) * (tileset.tileHeight + tileset.spacing) + tileset.margin;

float worldX = x * tileWidth;
float worldY = y * tileHeight;
```

Thông số kỹ thuật: Bản đồ định nghĩa rõ kích thước lưới (tileWidth, tileHeight), tạo cơ sở cho việc tính toán tọa độ thế giới (World Coordinates) của mọi thực thể.

3.3.2 Kỹ thuật Render tối ưu và Xử lý khung nhìn (Culling).

Lớp TiledMapRenderer chịu trách nhiệm chuyển đổi dữ liệu bản đồ thành hình ảnh trên màn hình với các kỹ thuật tối ưu:

```
int startX = Math.max(0, (int) (left / tileWidth) - 1);
```

```
    int endX = Math.min(map.getWidth() - 1, (int) (right / tileWidth) + 1);
    int startY = Math.max(0, (int) (bottom / tileHeight) - 1);
    int endY = Math.min(map.getHeight() - 1, (int) (top / tileHeight) + 1);
```

Thuật toán Culling (Cắt tỉa): Thay vì vẽ toàn bộ bản đồ khổng lồ, hệ thống tính toán các chỉ số startX, endX, startY, endY dựa trên vị trí hiện tại của Camera2D. Chỉ những ô gạch nằm trong hoặc mấp mé khung nhìn mới được đưa vào vòng lặp render, giúp duy trì tốc độ khung hình (FPS) ổn định ngay cả với bản đồ rất lớn.

```
float screenX = worldX - camera.x + camera.viewportWidth / 2f;
float screenY = camera.viewportHeight / 2f + camera.y - worldY - tileHeight;
```

Chuyển đổi hệ tọa độ: Phương thức tính toán screenX và screenY thực hiện phép trừ vị trí Camera khỏi tọa độ thế giới, đảm bảo các Tile được vẽ đúng vị trí tương quan khi người chơi di chuyển.

```
    AffineTransform transform = new AffineTransform(m00, m10, m01, m11, m02, m12);
    AffineTransform old = g.getTransform();
```

Trong hàm drawTile, thay vì sử dụng cách vẽ thông thường, lớp này sử dụng AffineTransform. Đây là kỹ thuật ma trận cho phép: Lật hình chính xác: Xử lý các trường hợp ô gạch được xoay hoặc lật trong trình biên tập bản đồ Tiled. Xử lý FlipD (Diagonal): Chuyển đổi các trục tọa độ để thực hiện phép quay 90 độ kết hợp lật, đảm bảo hình ảnh hiển thị đúng ý đồ thiết kế màn chơi.

```
 boolean flipH = (gidWithFlags & FLIP_H) != 0;
 boolean flipV = (gidWithFlags & FLIP_V) != 0;
 boolean flipD = (gidWithFlags & FLIP_D) != 0;
```

Một điểm đặc biệt trong mã nguồn là khả năng xử lý các ô gạch bị lật hoặc xoay trực tiếp từ dữ liệu Tiled . Giải mã Bitwise: Hệ thống sử dụng các mặt nạ Bit (FLIP_H, FLIP_V, FLIP_D) để kiểm tra trạng thái của GID. Bằng cách sử dụng các phép toán logic trên bit (& và ~), chương trình có thể tách biệt ID của ô gạch khỏi các cờ trạng thái lật hình.

Lớp nạp dữ liệu bản đồ TmxMapLoader:

Bên cạnh các lớp dữ liệu như TiledMap, TiledLayer, Tileset và bộ kết xuất TiledMapRenderer, hệ thống bản đồ của trò chơi còn phụ thuộc trực tiếp vào lớp TmxMapLoader để đọc và chuyển đổi dữ liệu từ tệp bản đồ định dạng .tmx thành các đối tượng mà chương trình có thể sử dụng. Đây là thành phần trung gian quan trọng kết nối giữa tài nguyên thiết kế trong phần mềm Tiled và cấu trúc dữ liệu nội bộ của game.

```
public class TmxMapLoader {
  public TiledMap load(String relativePath) {
    try {
```

```
        Path mapPath = Paths.get(System.getProperty("user.dir"), "assets", relativePath).normalize();
        Document doc =
DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(mapPath.toFile());
        doc.getDocumentElement().normalize();

        Element mapElement = doc.getDocumentElement();
        int width = Integer.parseInt(mapElement.getAttribute("width"));
        int height = Integer.parseInt(mapElement.getAttribute("height"));
        int tileWidth = Integer.parseInt(mapElement.getAttribute("tilewidth"));
        int tileHeight = Integer.parseInt(mapElement.getAttribute("tileheight"));

        // doc tileset va layer de tao doi tuong TiledMap
    } catch (Exception e) {
        throw new IllegalStateException("Failed to load TMX: " + relativePath, e);
    } }
```

Trong phương thức load(String relativePath), lớp TmxMapLoader thực hiện việc mở tệp .tmx, phân tích cú pháp XML và trích xuất các thông số cơ bản của bản đồ như số lượng ô theo chiều ngang, chiều dọc, kích thước từng ô gạch và danh sách các lớp bản đồ. Sau đó, hệ thống tiếp tục đọc thông tin tileset, xử lý đường dẫn tới các tệp .tsx, nạp ảnh tileset tương ứng và khởi tạo các đối tượng Tileset để phục vụ cho quá trình dựng hình.

Một nhiệm vụ quan trọng khác của lớp này là đọc dữ liệu layer ở dạng CSV và chuyển đổi chúng thành mảng gid dùng trong chương trình. Trong quá trình đó, dữ liệu từng hàng còn được đảo lại theo trục dọc để phù hợp với hệ tọa độ hiển thị của trò chơi. Nhờ vậy, dữ liệu do phần mềm Tiled xuất ra có thể được ánh xạ chính xác sang môi trường thực thi của game.

```
String csv = dataElement.getTextContent().trim();
String[] values = csv.split("[,\\s]+");
int[] gids = new int[layerWidth * layerHeight];
int index = 0;
for (int row = 0; row < layerHeight; row++) {
    int targetRow = layerHeight - 1 - row;
    for (int col = 0; col < layerWidth; col++) {
        long rawGid = Long.parseLong(values[index++].trim());
        int gidWithFlags = (int) rawGid;
        gids[targetRow * layerWidth + col] = gidWithFlags;
    }
}
```

3.4. Hệ thống Tiện ích và Quản lý Tài nguyên (Utilities & Asset Management)

Để tối ưu hóa hiệu suất và tái sử dụng mã nguồn, trò chơi xây dựng một bộ thư viện tiện ích dùng chung, giải quyết các vấn đề từ quản lý bộ nhớ đến xử lý đồ họa và âm thanh.

3.4.1 Quản lý Tài nguyên và Bộ nhớ (Assets System).

Lớp Assets đóng vai trò là một Resource Cache trung tâm :

```
public final class Assets {
    private static final Map<String, BufferedImage> CACHE = new HashMap<>();
    private static final Path ROOT = Paths.get(System.getProperty("user.dir"), "assets");

    private Assets() {}
    public static BufferedImage load(String relativePath) {
        return CACHE.computeIfAbsent(relativePath, Assets::loadInternal);
    }
    private static BufferedImage loadInternal(String relativePath) {
        Path path = ROOT.resolve(relativePath).normalize();
        if (!Files.exists(path)) {
            throw new IllegalStateException("Missing asset: " + path);
        }
        try {
            return ImageIO.read(path.toFile());
        } catch (IOException e) {
            throw new IllegalStateException("Failed to load asset: " + path, e);
        }
    }
}
```

Lớp Assets được thiết kế theo mô hình Static Utility Class kết hợp với cơ chế Caching nhằm tối ưu hóa việc quản lý tài nguyên hình ảnh trong trò chơi. Bằng cách sử dụng một HashMap làm bộ nhớ đệm, hệ thống đảm bảo mỗi tệp tin hình ảnh chỉ được tải từ ổ đĩa một lần duy nhất, từ đó giảm thiểu đáng kể chi phí truy xuất I/O và tiết kiệm tài nguyên RAM khi tái sử dụng cùng một hình ảnh nhiều lần. Phương thức load sử dụng computeIfAbsent để tự động kiểm tra: nếu hình ảnh đã tồn tại trong CACHE thì trả về ngay lập tức, nếu chưa sẽ kích hoạt logic loadInternal để đọc tệp từ đường dẫn tuyệt đối dựa trên thư mục gốc assets. Ngoài ra, lớp này còn tích hợp khả năng xử lý lỗi chặt chẽ, sẵn sàng ném ra ngoại lệ nếu tệp tin bị thiếu hoặc hỏng, giúp lập trình viên dễ dàng kiểm soát tính toàn vẹn của dữ liệu trong quá trình phát triển.

3.4.2. Hệ thống Hoạt ảnh và Đồ họa (Animation & GraphicsUtil).

```
public class Animation {
    private final BufferedImage[] frames;
    private final float frameDuration;

    public Animation(float frameDuration, BufferedImage[] frames) {
        this.frameDuration = frameDuration;
        this.frames = frames;
    }
    public BufferedImage getFrame(float stateTime, boolean loop) {
        if (frames.length == 0) {
            return null;
        }
        int frameIndex = (int) (stateTime / frameDuration);
        if (loop) {
            frameIndex = frameIndex % frames.length;
        } else {
            frameIndex = Math.min(frames.length - 1, frameIndex);
        }
        return frames[frameIndex];
```

```
    }
    public boolean isFinished(float stateTime) {
        return stateTime >= frameDuration * frames.length;
    }
}
```

Lớp Animation đóng vai trò là bộ khung quản lý các chuỗi hình ảnh chuyển động, cho phép trò chơi diễn đạt các hành động của nhân vật một cách mượt mà thông qua cơ chế xử lý khung hình theo thời gian.

Cấu trúc cốt lõi của lớp dựa trên một mảng các hình ảnh (frames) và thời gian hiển thị mỗi khung (frameDuration). Điểm mấu chốt nằm ở phương thức getFrame, nơi hệ thống tính toán chỉ số khung hình dựa trên tổng thời gian đã trôi qua (stateTime). Thuật toán này hỗ trợ linh hoạt hai chế độ: lặp lại vô tận (sử dụng phép chia lấy dư %) cho các hành động như chạy bộ, hoặc dừng ở khung hình cuối (sử dụng Math.min) cho các hiệu ứng chỉ diễn ra một lần. Cuối cùng, hàm isFinished cung cấp một công cụ kiểm tra trạng thái chính xác, giúp hệ thống logic của game biết khi nào một chuỗi hoạt cảnh (như tung đòn đánh hoặc tử vong) đã hoàn tất để chuyển sang trạng thái tiếp theo.

```
    public static void drawImage(Graphics2D g, BufferedImage image, float x, float y, float width, float height, boolean flipX, boolean flipY)
{
        if (image == null) {
            return;
        }
        AffineTransform transform = new AffineTransform();
        float drawX = x;
        float drawY = y;
        float scaleX = width / image.getWidth();
        float scaleY = height / image.getHeight();

        if (flipX) {
            scaleX = -scaleX;
            drawX = x + width;
        }
        if (flipY) {
            scaleY = -scaleY;
            drawY = y + height;
        }
        transform.translate(drawX, drawY);
        transform.scale(scaleX, scaleY);
        g.drawImage(image, transform, null);
    }
```

Phương thức drawImage này là một công cụ xử lý đồ họa quan trọng, cho phép hiển thị hình ảnh với các hiệu ứng co giãn và lật (flip) linh hoạt. Bằng cách sử dụng lớp AffineTransform, hệ thống thực hiện các phép biến đổi toán học lên ma trận hình ảnh thay vì chỉ vẽ các điểm ảnh tĩnh. Khi tham số lật (flipX hoặc flipY) được kích hoạt, mã nguồn sẽ đảo ngược tỉ lệ (scale) sang giá trị âm và điều chỉnh lại tọa độ gốc (drawX, drawY) để đảm bảo hình ảnh không bị hiển thị sai vị trí. Kỹ thuật này giúp

tiết kiệm tài nguyên bộ nhớ đáng kể vì lập trình viên có thể tạo ra các hướng nhìn khác nhau cho nhân vật .

3.4.3. Tiện ích Văn bản và Camera (TextUtil & Camera2D).

```
private TextUtil() {}

public static List<String> wrapText(FontMetrics metrics, String text, float maxWidth) {
    List<String> lines = new ArrayList<>();
    if (text == null || text.isEmpty()) {
        return lines;
    }
    String[] words = text.split("\\s+");
    StringBuilder current = new StringBuilder();
    for (String word : words) {
        if (current.length() == 0) {
            current.append(word);
            continue;
        }
        String candidate = current + " " + word;
        if (metrics.stringWidth(candidate) <= maxWidth) {
            current.append(" ").append(word);
        } else {
            lines.add(current.toString());
            current = new StringBuilder(word);
        }
    }
```

TextUtil: Hỗ trợ xử lý văn bản động, tự động tính toán độ dài chuỗi để ngắt dòng (wrapText) sao cho phù hợp với khung hội thoại trong MessageOverlay.

```
public class Camera2D {
    public float x;
    public float y;
    public float viewportWidth;
    public float viewportHeight;

    public Camera2D(float viewportWidth, float viewportHeight) {
        this.viewportWidth = viewportWidth;
        this.viewportHeight = viewportHeight;
    }
}
```

Camera2D: Lưu trữ trạng thái khung nhìn của người chơi. Đây là thành phần cầu nối giúp chuyển đổi giữa tọa độ thế giới thực tế và tọa độ hiển thị trên màn hình máy tính, tạo ra hiệu ứng di chuyển mượt mà của môi trường theo bước chân nhân vật.

3.5. Hệ thống Quản lý và Xử lý Âm thanh (Audio Engine)

Hệ thống âm thanh trong game được thiết kế không chỉ để tạo không khí mà còn đóng vai trò là một kênh phản hồi thông tin quan trọng tới người chơi. Hệ thống này được chia làm hai phân hệ chính dựa trên đặc tính kỹ thuật của Java Sound API.

3.5.1. Kiến trúc quản lý âm thanh đa tầng.

Để tối ưu hóa bộ nhớ và hiệu suất xử lý, âm thanh được phân loại dựa trên mục đích sử dụng:

```
public final class LoopingSound {
    private static final Path ROOT = Paths.get(System.getProperty("user.dir"), "assets");

    private Clip clip;
    public void play(String relativePath) {
        stop();
        Path path = ROOT.resolve(relativePath).normalize();
        if (!Files.exists(path)) {
            throw new IllegalStateException("Missing sound: " + path);
        }
        try (AudioInputStream stream = AudioSystem.getAudioInputStream(path.toFile())) {
            clip = AudioSystem.getClip();
            clip.open(stream);
            clip.loop(Clip.LOOP_CONTINUOUSLY);
            clip.start();
        } catch (IOException | LineUnavailableException | UnsupportedAudioFileException e) {
            throw new IllegalStateException("Failed to play sound: " + path, e);
        }
    }
    public void stop() {
        if (clip == null) {
            return;
        }
        clip.stop();
        clip.close();
        clip = null;
    }
}
```

Âm thanh nền (Background Audio): Sử dụng lớp LoopingSound. Cơ chế này tải tệp âm thanh vào một Clip, thiết lập chế độ lặp vô hạn (LOOP_CONTINUOUSLY). Đây là giải pháp cho nhạc nền (BGM) và âm thanh môi trường (BGS), đảm bảo trải nghiệm không bị ngắt quãng khi người chơi di chuyển trong rừng Emerald.

```
public final class OneShotSound {
    private static final Path ROOT = Paths.get(System.getProperty("user.dir"), "assets");

    private OneShotSound() {}
    public static void play(String relativePath) {
        Clip clip = playClip(relativePath);
        if (clip != null) {
            clip.start();
        }
    }
    public static Clip playClip(String relativePath) {
        Path path = ROOT.resolve(relativePath).normalize();
        if (!Files.exists(path)) {
            throw new IllegalStateException("Missing sound: " + path);
        }
        try {
            AudioInputStream stream = AudioSystem.getAudioInputStream(path.toFile());
            Clip clip = AudioSystem.getClip();
            clip.addLineListener(event -> {
                if (event.getType() == LineEvent.Type.STOP) {
                    clip.close();
```

```
            try {
                stream.close();
            } catch (IOException ignored) {}
        }
    });
    clip.open(stream);
    return clip;
  } catch (IOException | LineUnavailableException | UnsupportedAudioFileException e) {
    throw new IllegalStateException("Failed to play sound: " + path, e);
  }
}
public static void stop(Clip clip) {
  if (clip == null) {
    return;
  }
  if (clip.isRunning()) {
    clip.stop();
    return;
  }
  clip.close();  } }
```

Hiệu ứng tức thời (Sound Effects - SFX): Sử dụng lớp OneShotSound. Các âm thanh như tiếng vung kiếm, tiếng quái vật bị thương hay tiếng bước chân được xử lý theo cơ chế "phát và giải phóng".

Cơ chế tự động giải phóng tài nguyên thông qua LineListener là một giải pháp quản lý bộ nhớ thông minh, giúp hệ thống âm thanh vận hành ổn định trong môi trường đa luồng. Thay vì để các đối tượng âm thanh tồn tại vô hạn sau khi phát xong, lớp OneShotSound thiết lập một bộ lắng nghe sự kiện trực tiếp trên dòng âm thanh. Ngay khi hiệu ứng kết thúc—tương ứng với sự kiện LineEvent.Type.STOP—hệ thống sẽ lập tức kích hoạt lệnh clip.close() và stream.close(). Quy trình này đảm bảo các đường truyền âm thanh (Lines) và luồng dữ liệu tệp tin được đóng lại kịp thời, ngăn chặn tình trạng rò rỉ bộ nhớ (Memory Leak) và lỗi cạn kiệt tài nguyên hệ thống, đặc biệt là trong các phân cảnh giao tranh dồn dập với hàng loạt hiệu ứng cháy nổ hay va chạm diễn ra liên tục.

3.5.2. Hệ thống phản hồi thính giác theo trạng thái (Audio-Visual Sync).

Âm thanh được lập trình để đồng bộ hóa tuyệt đối với hệ thống Máy trạng thái (FSM) của thực thể:

```
private void updateRunSound(float delta) {
  boolean shouldPlayRunStep = onGround && !isAttacking && Math.abs(velocityX) > 1f;
  if (!shouldPlayRunStep) {
    runStepTimer = 0f;
    return;
  }
  runStepTimer -= delta;
  if (runStepTimer <= 0f) {
    playRunSound();
    runStepTimer = RUN_STEP_INTERVAL;
  }
}
```

Đồng bộ hoạt ảnh: Tiếng bước chân được tính toán thông qua runStepTimer trong lớp Player, đảm bảo âm thanh phát ra đúng khoảnh khắc chân nhân vật chạm đất trên Sprite Sheet.

```
private void startAttack(int stage, InputState input) {
    isAttacking = true;
    attackSerial++;
    attackStage = Math.max(1, Math.min(stage, 3));
    queuedAttackStage = attackStage;
    attackWithMovement = onGround && (input.isKeyDown(KeyEvent.VK_A) || input.isKeyDown(KeyEvent.VK_D));
    stateTime = 0f;
    playAttackSound(attackStage);
}
```

Phương thức startAttack(int stage, InputState input): Đây là nơi tiếp nhận giai đoạn tấn công hiện tại (attackStage). Khi một đòn đánh mới bắt đầu, nó sẽ gọi hàm hỗ trợ để phát âm thanh tương ứng. Dựa vào biến stage (1, 2 hoặc 3), nó sẽ gọi lớp tiện ích OneShotSound để phát các tệp như Sword Attack 1.wav, Sword Attack 2.wav, hoặc Sword Attack 3.wav.

Lời Kết

Qua quá trình thực hiện đề tài xây dựng game 2D side-scrolling bằng Java, chúng em đã áp dụng được các kiến thức về lập trình hướng đối tượng, xử lý đồ họa, các kỹ thuật SWING/AWT, cũng như quản lý sự kiện trong môi trường phát triển game. Sản phẩm tuy đơn giản nhưng đã thể hiện được các chức năng cơ bản của một trò chơi như điều khiển nhân vật, xử lý va chạm, di chuyển màn hình và tính điểm.

Trong quá trình thực hiện, nhóm cũng gặp không ít khó khăn như tối ưu hiệu năng, xử lý logic game mượt mà và thiết kế cấu trúc chương trình hợp lý. Tuy nhiên, thông qua việc tìm hiểu tài liệu và thử nghiệm nhiều cách tiếp cận khác nhau, chúng em đã từng bước khắc phục và hoàn thiện sản phẩm.

Đề tài không chỉ giúp củng cố kiến thức đã học mà còn mở ra cơ hội tìm hiểu sâu hơn về phát triển game, đặc biệt là các kỹ thuật nâng cao như animation, di chuyển đơn giản và tối ưu trải nghiệm người dùng. Trong tương lai, nếu có điều kiện, chúng em mong muốn phát triển thêm các tính năng như âm thanh, nhiều màn chơi hơn và cải thiện giao diện để trò chơi trở nên hấp dẫn hơn.

Chúng em xin chân thành cảm ơn sự hướng dẫn và hỗ trợ từ giảng viên để hoàn thành đề tài này.

TÀI LIỆU THAM KHẢO

Slide bài giảng của giảng viên Đào Thị Lệ Thủy – ĐH GTVT
Tham khảo tutorial, file handling, data structure các classes, method và các hướng dẫn khác liên quan chi tiết tại:
https://www.w3schools.com/java/default.asp

Tham khảo một số video bài giảng java của MrJavaHelp:
https://www.youtube.com/watch?v=zCRgJAHWEYA
https://www.youtube.com/watch?v=NIUKHHVF9ek
Tham khảo cơ chế của trò chơi qua series How to make 2d Game in Java:
https://www.youtube.com/playlist?list=PL_QPQmz5C6WUF-pOQDsbsKbaBZqXj4qSq